University of Science and Technology of Hanoi

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Group Project Report

Tax Legal Retrieval with
Graph Representation Assistant [EN]

Authors:

Pham Duy Anh	ID - 23BI14023
Nguyen Thai Nhat Anh	ID - 23BI14024
Le Quoc Anh	ID - 23BI14025
Dinh Thanh Hieu	ID - 22BA13132
Ngo Xuan Kien	ID - 23BI14239
Nguyen The Khai	ID - 23BI14205

Supervisor: Tran Giang Son – ICT Department

HANOI, 2025

Acknowledgement

This project was strongly influenced and supported by Professor Tran Giang Son, the Director of Department of Information and Communication Technology, University of Science and Technology of Hanoi (ICT-USTH). We would like to express our sincere gratitude for his thoughtful guidance, valuable recommendations, and continuous support throughout the course of this project. Without his sustained involvement and encouragement, this work would not have been completed to its present extent.

Abstract

Retrieval-Augmented Generation (RAG) has become a popular, widely-applied approach for grounding Large Language Models (LLM) with external knowledge. However, traditional RAG systems primarily rely on flat, stateless text chunking without hierarchical awareness, which struggles to capture the semantic structure and dense cross-referencing network in legal documents, especially Vietnamese official legal documents.

In this work, we propose a comprehensive end-to-end framework for graph-integrated context retrieval over a corpus of Vietnamese legal documents in the domain of fees and taxation. The proposed pipeline consists of document collection, hierarchical graph construction with explicit cross-document references, and the generation of a tailored question–answer benchmark dataset for graph-vector retrieval comparison. We evaluate multiple retrieval strategies by comparing graph-based retrieval methods with traditional vector-based similarity retrieval to evaluate our hypothesis regarding the effectiveness of graph-integrated retrieval in the Legal document system.

Experimental results indicate that while the proposed graph-based approach does not yet consistently outperform vector retrieval in all settings, it provides improved interpretability, citation traceability, and document structural awareness. These findings highlight the potential of graph representations to address fundamental limitations of traditional legal RAG systems and suggest directions for future improvements.

Contents

1 Introduction **7**
1.1 Context and Background . . . 7
1.2 Motivation . . . 7
1.3 Objectives . . . 8

2 Literature Review **8**
2.1 RAG approaches in the Legal domain . . . 8
2.2 Knowledge Graph integrated RAG . . . 8
2.3 Report Structure . . . 9

3 Materials & Methods **10**
3.1 Document Collection . . . 10
3.2 Document Graph Representation . . . 12
3.3 QA Benchmark Dataset . . . 24
3.3.1 Proposed data model . . . 24
3.3.2 Crawled FAQ and Generated dataset . . . 25
3.4 Retrieval Pipeline and Graph-Integrated Retrieval . . . 28
3.4.1 Vector Retrieval . . . 28
3.4.2 Graph Retrieval . . . 29
3.5 Demo UI Visualization . . . 34

4 Results & Discussion **36**
4.1 Evaluation Metrics . . . 36
4.2 Results . . . 37
4.3 Discussion . . . 37

5 UI Visualization **38**

6 Conclusion & Future Work **39**

List of Figures

1 Overall Methodology Pipeline . . . 10
2 Document corpus Distribution . . . 11
3 Graph Representation Pipeline . . . 12
4 Document Metadata Extraction Model . . . 14
5 Training Relation Distribution for RE task . . . 15
6 Sample Lexicon masking, where the index 2-3 and 16 (index of keywords ”Nghị định” and ”Luật”) receive the value of 1 while others are 0.1 . . . 17
7 Document Relation Extraction Model . . . 19
8 Document Cross-References appear at the start of the document . . . 20
9 Resulting Cross-Document References, note that only 3 connections are set up due to the recognition of Document ID . . . 21
10 Example Document Self References, as the node ”*Khoản 3*” mentions ”*Khoản 2, 3 của Điều này*”, a Self-loop and an Edge will be constructed . . . 24
11 LLM generated QA dataset Data model . . . 26
12 Retrieval pipeline . . . 28
13 Example of the Chunked Hierarchy method, where a Node and its child components are combined into 1 Chunk node connected by IS_IN relationship . . . 30
14 GraphSAGE pipeline . . . 31
15 ComplEx Embedding pipeline with Gated Fusion . . . 32
16 Sample Graph Traversal, the system first retrieves Node 2 then collects the top 5 nodes within 2 hops neighboring nodes (0, 1, 3, 4, 5) ranked by node ID and concatenates all of the text into 1 chunk . . . 33
17 UML Use case Diagram . . . 34
18 UI Sequence Diagram . . . 35
19 User interface overview . . . 39

List of Abbreviations

Abbreviation	Meaning
FAQ	Frequently Asked Questions
RAG	Retrieval-Augmented Generation
IE	Information Extraction
NER	Named Entity Recognition
RE	Relation Extraction
QA	Question-Answering
POStag	Part-of-Speech Tagging
BERT	Bidirectional Encoder Representation from Transformer
CRF	Conditional Random Field
LN	Layer Normalization
PE	Positional Encoding
GDS	Graph Data Science

List of Legal Words

Abbreviation	Meaning
Constitution	Hiến Pháp
Law	Luật
Decree	Nghị Định
Resolution	Nghị Quyết
Circular	Thông Tư
Decision	Quyết Định
Regulation	Quy Chế
Directive	Chỉ Thị
Chapter	Chương
Clause	Điều
Point	Khoản
Subpoint / Subsubpoint / Subsubsubpoint	Điểm

1 Introduction

1.1 Context and Background

Legal documents, in general, are official instruments issued by government authorities to formalize legal agreements, rights, and obligations within a country, serving as evidence that can be enforced in a court of law and should be understood by all citizens. Vietnam follows a civil law system with a strict and well defined hierarchy of documents, this legal framework is made up of multiple document types issued by different authorities, including Constitution (Hiến pháp), Laws (Luật) or Resolutions (Nghị quyết),... Each document type has a different legal role and must be interpreted in relation to higher and lower level documents in the administrative order.

While this hierarchical structure provides conceptual clarity, it also leads to a large and highly fragmented legal corpus where legal statements frequently reference other documents, and clauses within a single document often refer to other articles or clauses, resulting in dense cross-referencing throughout the system. Consequently, both legal professionals and non-expert users face significant challenges in efficiently locating relevant, valid, and up-to-date information, particularly when navigating across multiple levels of the legal hierarchy both within and between documents.

1.2 Motivation

While Large Language Models (LLMs) show strong potential for summarizing legal texts and answering legal questions, direct use of these models often leads to problems with factual accuracy, missing citations and lack of source traceability. Retrieval-Augmented Generation (RAG) helps address these issues by grounding model outputs in provided external legal sources. However, traditional RAG systems treat legal documents as flat text collections, where all documents and passages are indexed independently at the same level without preserving their legal role or position in the hierarchy [14]. This approach ignores important legal structure and cross-references between documents and documents' clauses. As a result, retrieval is based only on surface, individual disconnected text chunks, making it difficult to support structured legal reasoning or produce verifiable, legally valid citations [13].

These limitations have motivated the exploration of graph representations that explicitly model legal hierarchies and inter-document relationships. By encoding document levels and cross-node edges, recent proposed approaches design a Graph representation of legal documents based on Natural Language Processing (NLP) and the assistance of LLMs for graph construction. However, we identify a notable research gap in existing work on graph-integrated retrieval for legal documents. Most comprehensive and recent approaches to graph-based retrieval, such as [28] and [38], are developed and evaluated primarily for non-Vietnamese languages, and their assumptions may not directly apply to the Vietnamese legal context. Conversely, existing Vietnamese studies in this area remain limited in scope. For instance, [26] focuses specifically on legal statute cases without addressing broader legal document types, while [27], despite constructing a detailed and thorough graph representation of Vietnamese legal documents, does not focus on extending the resulting graph to support graph-integrated embeddings or retrieval strategies.

Our proposed approach addresses this research gap by inheriting and extending graph representation strategies from prior works and applying graph-based embedding and information retrieval methods explicitly fostered by graph integration. Moreover, we construct our own customized training dataset and deliberately minimize dependence on black box LLMs, instead focusing on task-specific Deep Learning models for information extraction and graph retrieval strategies.

1.3 Objectives

The main objective of this project is to use graph-based representations and related strategies to improve information retrieval for Vietnamese legal documents, specifically, we focus on proposing an end-to-end framework for Graph representation of legal document systems, integrating extraction of document's metadata, modeling of hierarchical node structures, and relational edges that capture both structural hierarchy and inter-document dependencies.

Apart from the database structuring, a ”real FAQ”-inspired QA dataset will be built to act as a benchmark evaluator for Graph-Vector comparison, which can be achieved from applying different approaches for Graph relational embedding and Graph-assisted retrieval to compare the information retrieval performance over traditional approaches.

2 Literature Review

2.1 RAG approaches in the Legal domain

RAG systems have been widely adopted in document understanding and question answering applications, including the legal domains. Prior works have demonstrated the effectiveness of Legal-adapted RAG approaches, such as the study by [8], which proposes a RAG framework tailored for Korean legal documents, or the work of [9], which enhances retrieval performance by incorporating structured summaries extracted from Indian legal cases. Moreover, several studies aim to improve retrieval effectiveness for general legal documents by addressing domain-specific challenges. For instance, [14] improves retrieval accuracy by embedding legal content and formal identifiers separately, enabling more precise grounding of legal references. Similarly, [11] utilizes Legal Named Entity Recognition (NER) to detect missing or ambiguous references.

However, applying RAG to the Vietnamese legal system introduces additional complexity due to the strict hierarchical organization of legal documents and language-specific tokenization features. To address these challenges, several studies have proposed enhancements to data preprocessing and retrieval strategies. For example, [12] improves retrieval performance through refined data processing and retrieval fusion, combining semantic search, hybrid retrieval strategies, and re-ranking methods. Similarly, [19] optimize multistage retrieval pipelines to dynamically improve retrieval quality.

Despite these advances, existing approaches operate on flattened text chunks and remain limited in their ability to explicitly interpret hierarchical structure. In particular, they lack awareness of the hierarchical position of retrieved contexts, do not incorporate cross-document or cross-clause references, and provide limited support for transparent citation in the generation phase [38] [13] [14]. These limitations have motivated the adoption of graph-based representations.

2.2 Knowledge Graph integrated RAG

In recent years, multiple studies have tackled the challenge of traditional RAG framework by introducing Graph integration, which facilitates the use of graph's connected structure for comprehensive information retrieval and avoid the context fragmentation of traditional RAG chunking [38]. Recent works such as [29], [32] or [26] along with the introduced LLM Graph Builder feature from Neo4j define an entity-level knowledge graph by extracting individual semantic entities and their relationships from the textual context, resulting in a diverse, heterogeneous set of entities with distinct attributes and inter-connected by semantic edges.

However, applying this method on documents with strong contextual significance and dense citation structure like Legal documents is proved impractical for exhaustive extraction of all possible individ-

ual entities and their relationships, thus not only overcomplicating the problem formulation and model training process [41] [40]. As an alternative approach, recent practices in document graph representation focus on preserving the hierarchical organization of a document across multiple tree levels [38] while leveraging passage-level entity and relation recognition to highlight cross-document relationships that enrich the existing graph structure [27] [28] [2].

Recent studies focus on identifying effective methods for encoding these rich graph structures into node embeddings to support comprehensive node/ subgraph retrieval or other downstream tasks. The work of [29] evaluates three graph embedding techniques - Wikipedia2Vec, RDF2Vec, and ComplEx - to model different types of entity-relation semantics. [35] introduces the GraphSAGE algorithm, a convolutional graph embedding method developed as an inductive extension of Graph Convolutional Networks (GCNs) originally proposed by Kipf and Welling [36] [37] tailored for Graph classification tasks.

Retrieval methods leveraging graph-facilitated features have also been explored in related studies. In particular, [31] introduces a Semantic Compression technique, which partitions knowledge graph nodes into clusters and penalizes in-cluster retrieval during graph traversal. The work also explores Personalized PageRank (PPR) as a graph traversal strategy for searching related nodes using the transition probabilities between neighboring nodes. Another study published in 2024 by [39] proposes integrating a Graph Neural Network (GNN) to combine semantic similarity scores between an input query and passage nodes with the positional relationships among these nodes to rank the top-k nodes based on this integrated information.

Our work focuses on adapting and extending such mentioned techniques to suit the specific requirements of the proposed tasks, with an emphasis on Vietnamese tax legal documents in particular and Vietnamese legal documents in general. The proposed approach deliberately excludes the use of large language models and instead relies on conventional NLP-based text processing techniques implemented within a deep learning framework to address the problem.

2.3 Report Structure

The subsequent sections of this report are organized as follows. The section 3 ”Materials & Methods” presents the proposed end-to-end pipeline, including legal document corpus collection, document graph representation, and the applied context retrieval approaches for the resulting document KG along with our Demo UI for pipeline visualization. The section 4 ”Results & Discussion” details the experimental configurations, evaluation metrics, and experiment results, the section also includes our discussion and analysis of the results along with current limitations. These findings and limitations will again be mentioned along with future improvements in the section 5 ”Conclusion & Future Works”

3 Materials & Methods

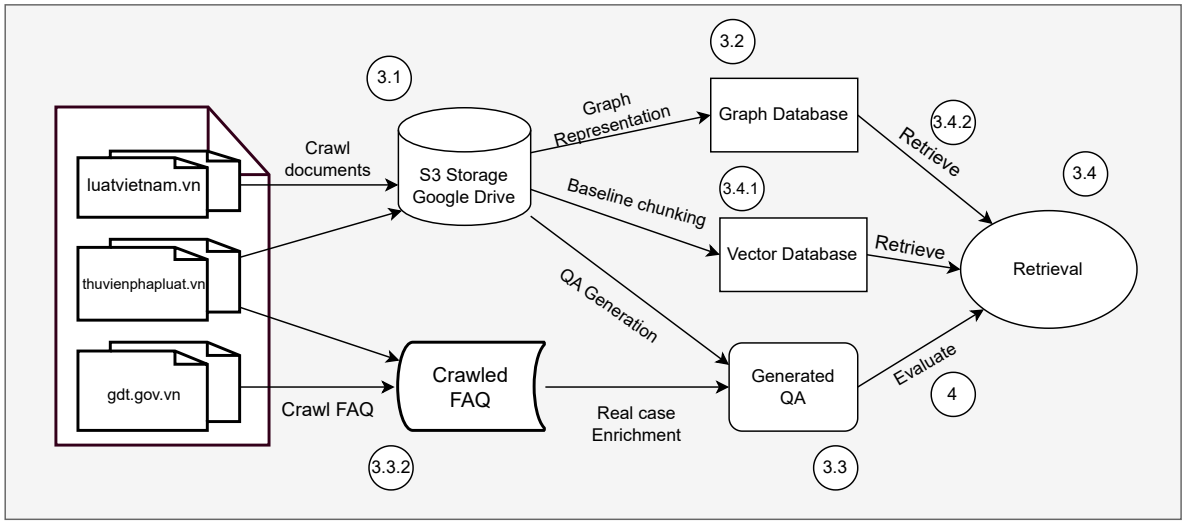


Figure 1: Overall Methodology Pipeline

3.1 Document Collection

We started the pipeline by defining a fixed collection of Vietnamese legal documents. We designed the data collection pipeline to crawl and extract Vietnamese legal documents from LuatVietnam.vn, a comprehensive legal document portal. The pipeline operates in two sequential stages:

URL listing

The system employs a Playwright-based headless browser to navigate the paginated AJAX search results. The crawler queries the search endpoint with specific document type identifiers (DocTypeIds), iterating through up to 2,000 pages per document type. The search URL follows the pattern:

”https://luatvietnam.vn/van-ban/ajax/searchajax? Keywords=&DocTypeIds={type_id}&SearchOptions=1 &RowAmount=20&PageIndex={page}”

Where the parameters specify:

- **Keywords**, this field is left empty to collect all documents belonging to the specified document type
- **DocTypeIds**: The supported predefined document identifiers, up to 18 different types are supported by the crawler but we only utilized 8 specific types

DocTypeId	Vietnamese	English
7	Hiến pháp	Constitution
10	Luật	Law
58	Bộ luật	Legal Code
11	Nghị định	Decree
13	Nghị quyết	Resolution
21	Thông tư	Circular
4	Chỉ thị	Directive
14	Pháp lệnh	Ordinance

Table 1: Supported Document Type Identifiers

- **SearchOptions** = 1, a flag enabling the default search behavior
- **RowAmount** = 20, indicating 20 document per queried page
- **PageIndex**, indicates the current page number

The resulting document URLs were saved to the respective folders of corresponding document types.

Content fetching

The second stage processes the collected URLs to download the full HTML content. For each URL, the fetcher navigates to the document page using a headless Chromium browser, extracts JSON-LD structured metadata embedded in the page and saves both the page content (used for text processing) and the document metadata (used for downstream filtering tasks) to the document file storage.

The resulting raw document corpus were saved in respective folder of 13 different types and a total of roughly 20,000 documents spanning from 2010 to 2025 of different legal fields. To align with the scope of the project, we applied a custom filter to select only Tax/Fee-related documents of specific types - Law, Resolution, Decree, Circular, and Decision - from the past five years (2020-2025), temporarily excluding amending documents as these will only help in the Document node versioning feature. The resulting corpus consists of 165 distinct documents of 5 different types.

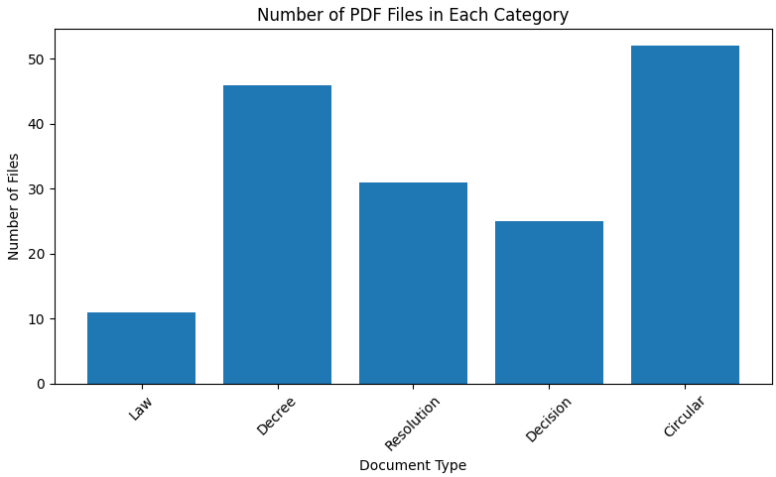


Figure 2: Document corpus Distribution

3.2 Document Graph Representation

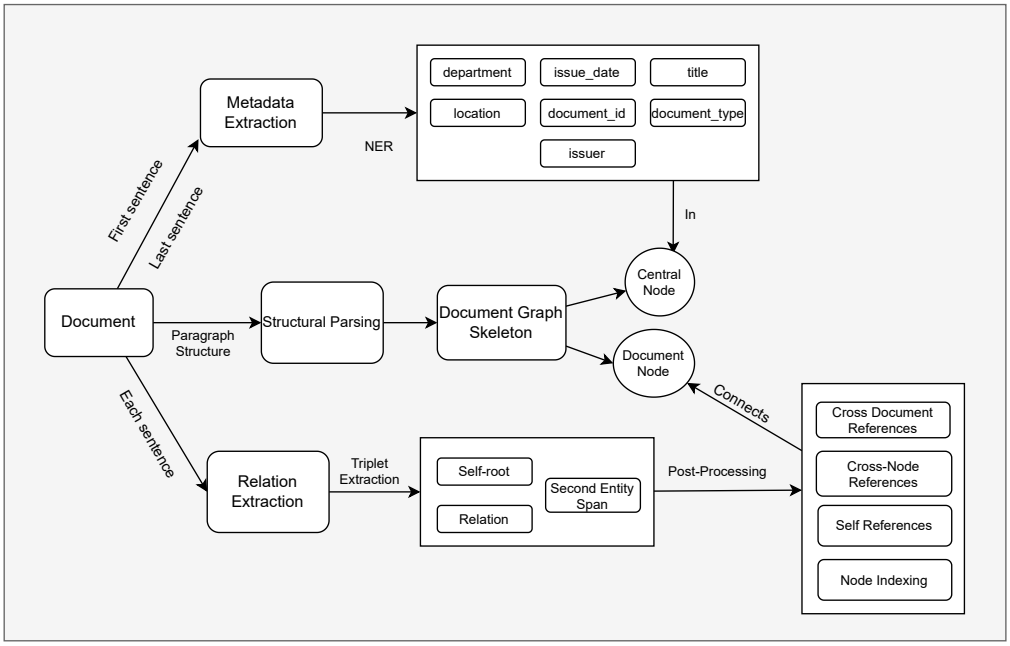


Figure 3: Graph Representation Pipeline

Structural Parsing

For the Graph Representation task, we followed the pipeline from recent researches [27] [28] to firstly construct the central Document node and traversed down based on the document hierarchical structure (Document → Chapter → Clause → Point → Subpoint → Subsubpoint → Subsubsubpoint)

Structure Level	Pattern
Document	Whole Document
Chapter	Chương 1
Clause	Điều 1
Point	Khoản 1
Subpoint	Điểm a)
Subsubpoint	Điểm a.1)
Subsubsubpoint	Điểm a.1.1)

Table 2: Document Structural Patterns

Where: for higher-level node label a and its lower component label b, we assigned the relationship

$$a - [HAS_b] \rightarrow b$$

For example:

Chapter_I – [HAS_CLAUSE]→Clause_1

The resulting initial representation is a simple graph where the Nodes are connected to each other based on hierarchical structure.

To account for the Node attributes, we first assigned the ”text” attribute from all content up to the next node in the hierarchy and assigned the ”id” attribute by a structured syntax of

$$node.id = ”DocumentID_chapter__C__P__SP__SSP__SSSP_”$$

For example, the Node starting with ”**Điều 3**” inside **”Chương 4”** of the Document **”Luật Doanh nghiệp số AB/CD/QH”** will be assigned the Node ID of ”**AB/CD/QH_chapter_IV_C_3**”

The Node attribute extraction follows a sequential and continuous process, where the system processes the text line by line while storing the higher-order structural hierarchy in a stateful dictionary for position retrieval. This approach ensures accurate identification of each node’s ID and its corresponding text at the cost of increased computation due to the sequential nature of processing. However, from the research about different Vietnamese legal document types, we have learnt that each type of document follows a different hierarchical structure, we later narrowed down the scope to a specific set of most relevant document types consisting of Law (following the proposed structure) and Decree/ Resolution/ Circular/ Decision (with no Chapter level) to simplify the extraction process (from which we applied the filter to the original document corpus).

Metadata Extraction

To capture the Document’s information such as **Document Type, Issued Location** or **Issue Date**, a model was built revolving around the Named Entity Recognition (NER) algorithm. Although extensive research has been conducted on NER for either legal documents or Vietnamese text, identifying a specific research direction that intersects these two domains to align with our initial scope remains a challenge. Without any officially available dataset, we created our own annotated dataset by taking a sample of 80 Legal documents of different types (Law, Decrees, Circulars,..), extracted the first and the last sentence (containing the related information) and manually annotated the sentences using the self-defined labels on the locally-hosted Label-studio application.

The Labeling process follows the predefined patterns, due to our greater emphasis on detecting the Document’s ID and Date attributes, some document titles containing the ID and Date such as ”**Nghị định Sửa đổi bổ sung Nghị định số 23/BI/NĐ-CP ngày 1 tháng 2 năm 2025 về vấn đề ....**” were labeled in a fragmented manner, which caused the model to miss the document title in certain cases.

Information Type	Sample Value	Label
Title	Luật Thu nhập doanh nghiệp	TIT
Document_ID	23BI/12/NĐ-CP	DOCID
Document_Type	Luật	TYP
Issuer Department	Chính Phủ	DEP
Issuer_Date	ngày 13 tháng 6 năm 2020	DAT
Issuer_Location	Hà Nội	LOC
Issuer	Phạm Minh Chính	PER

Table 3: NER labeling strategy

Following the work from [15], we formulated a Single forward model architecture with an input embedding layer followed by 2 **Bidirectional LSTM** layers for fully capturing the semantic meaning of the input sentence. Following that a Feed-Forward layer with Pre Layer-Normalization and Attention mask are applied and feed the final result through a last **Conditional Random Field** designed for the sequence tagging tasks.

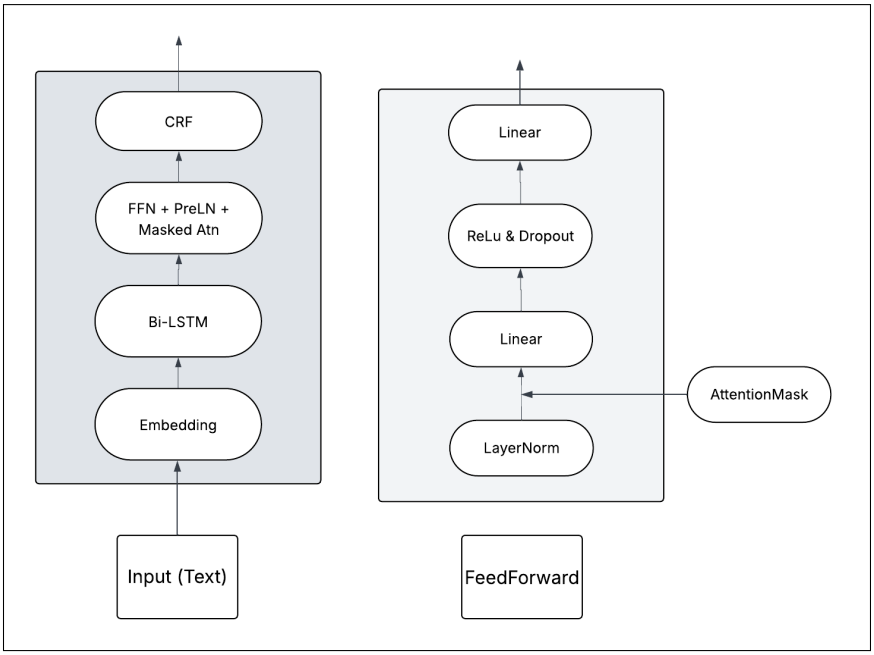


Figure 4: Document Metadata Extraction Model

The Model prediction was returned in the form of BIO (Begin - Inner - Outer) tagging scheme, which follows the rule:

```
Actual: "Ngay 13 thang 6 nam 2025" (DAT)
Prediction:
  - Ngay (B-DAT)
  - 13 (I-DAT)
  - thang (I-DAT)
  - 6 (I-DAT)
  - nam (I-DAT)
  - 2025 (I-DAT)
  - . (O)
```

We applied a set of post-processing steps to merge characters of the same type into a single block and enforced strict rules to prevent issues such as word fragmentation or case inconsistency. The resulting information was then automatically added as attributes of the central document node at the beginning of the processing phase.

department	issue_date	title	location	document_id	issuer	document_type
Chính Phủ	13/07/2020	Nghị định thay đổi…	Hà Nội	23/2020/NĐ-CP	Phạm Minh Chính	Nghị Định

Table 4: Sample Document Metadata Extraction

Relation Extraction

Along with the node attributes, we enriched the graph structure by defining several types of relational edges connected between nodes, each serving different purposes and use cases. Apart from the structural edges defined above, we split the relational edges into 3 different types.

Relation Types	Example	Relation Head	Possible Relation Labels	Relation Tail
Document Cross-Referencing	Căn cứ Luật Giá số 23BI/QH	Whole Document	Pursuant / Amended	Whole Document
Document Self-Referencing	Các nhân tố đề cập tại Điều 1 Luật này được quy định…	Document Node	Reference / Guide	Document Node (of same document)
Node Referencing	Bãi bỏ các Điều 32, 33, 34, 51, 138 và 139, điểm b khoản 1 Điều 163, khoản 3 và khoản 4 Điều 179 của Luật thi hành án dân sự số 26/2008/QH12.	Document Node	Reference / Guide / Amended / Replaced / PartReplace / Drop / PartDrop / Pursuant	Document Node (of other document)

Table 5: Relation Types and Attributes

For the relation extraction task, we employed the publicly available dataset introduced by [2], which formulates the task as the extraction of both the referenced document type and the corresponding relation type. These elements are pre-annotated in the source documents using an XML-based annotation scheme. The dataset comprises 5,031 XML–text formatted files, each containing annotated sentences that represent relational instances for extraction.

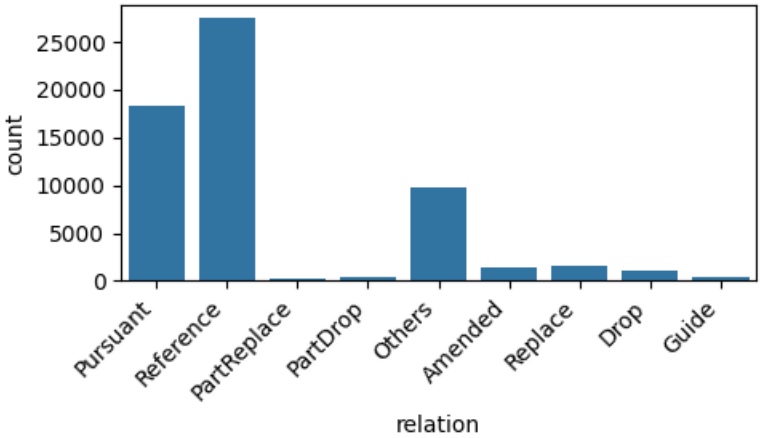


Figure 5: Training Relation Distribution for RE task

However, upon inspecting the annotations, we identified a major challenge for our initial approach: the dataset lacks explicit annotation for the first entity (relation head). To address this, we reframed the problem by extending the output space to include prediction of a Self-root flag, indicating whether the first entity corresponds to the entire document or to an individual document node, we also applied the Copy Mechanism from [4] to predict the Start/ End index of the second entity directly from the input sentence instead of generating one. Additionally, we removed all secondary and tertiary relations within each sentence, retaining only a single relation–entity pair per sentence to stabilize model predictions.

The final training dataset consists of 32,380 input sentences. Each sentence is paired with: (1) a binary self-root flag (0/1), (2) the relation type, and (3) the start and end indices of the second entity.

Input Text	Self-Root	Relation	Second Entity	Start Index	End Index	Relation Type
Luật này thay thế Luật hàng không dân dụng Việt Nam ngày 26 tháng 12 năm 1991.	1	Replace	Luật hàng không dân dụng Việt Nam ngày 26 tháng 12 năm 1991	4	16	Cross-Document Reference
Bãi bỏ các Điều 32, 33, 34, 51, 138 và 139, điểm b khoản 1 Điều 163, khoản 3 và khoản 4 Điều 179 của Luật thi hành án dân sự số 26/2008/QH12.	0	Drop	Điều 32, 33, 34, 51, 138 và 139, điểm b khoản 1 Điều 163, khoản 3 và khoản 4 Điều 179 của Luật thi hành án dân sự số 26/2008/QH12	3	33	Cross-Node Reference
Quyết định của Ủy ban nhân dân phường được ban hành để thực hiện chủ trương quy định tại khoản 1 Điều này và thực hiện chủ trương, biện pháp khác.	0	Reference	khoản 1 Điều này	21	24	Self-Reference

Table 6: Sample Relation Extraction Training Data (The ”Relation Type” and ”Second Entity” features are just for visualization

For the training of the RE model, we applied an Encoder-Decoder architecture based on the Transformer architecture proposed by [17] with our own set of layer modifications and the addition of semantic features. Inspired by the Joint-RE model approach from [1], we added a Lexicon Mask, a POStag Processing layer and Relative Positional Embedding to the Multi-head Self-Attention Logit instead of the traditional Sinusoidal Positional Encoding from the original Transformer architecture.

Lexicon Masking: Relations in legal documents usually involve the existence of certain legal entities such as:

- *”Luật, Nghị định, Nghị quyết,...”* for document reference
- ”*Điều, khoản, điểm,...*” for node reference

Based on the idea from [1], we built a custom lexicon mask matrix that assigns more weight to the legal entities, helping the model focus more on the entity recognition.

Given a lexicon list l_l and an input sequence a = $[a_1, a_2, a_3, ..., a_n]$

$$a_{ij} = \begin{cases} 1, & i = j, a_i \in l_l, \\ 1, & i = j, [a_i, a_{i+1}] \in l_l, \\ 1, & (i, j) = (i, i+1)|(i+1, i), \\ 0.1, & otherwise \end{cases}$$

Here we define the binary separation of 1 (lexicon token) and 0.1 (non-lexicon) instead of 1-0 so that non-lexicon tokens can still attend into the understanding of tokens instead of being excluded.

For example, an input sentence "***Ban hành Nghị định 23/2023/NĐ-CP sửa đổi bổ sung Luật đất đai***" will result in a lexicon mask:

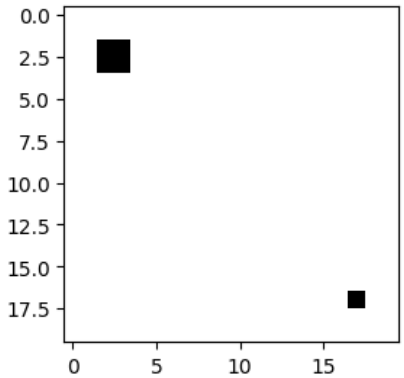


Figure 6: Sample Lexicon masking, where the index 2-3 and 16 (index of keywords "Nghị định" and "Luật") receive the value of 1 while others are 0.1

POStag Processing: We also tried to develop our own extracted feature bias, the Part-of-Speech tagging (POStag) processing. The **VnCoreNLP** module proposed by [3] shows its strength in Word Segmentation and POStagging for Vietnamese, which connects each subword token to the pretrained Part-of-speech label. Utilizing these features, we created a sequence of POStags with the same length as the input sentence.

For example:

Input: "***khoản 1 và điểm a khoản 2 Luật thi hành án dân sự số 26/2008/QH12***"

Output: "***N M Cc N a N M N V N N N N M***"

These sets of POS tags are mapped to an index dictionary to obtain a numerical representation. To model the interrelations between tokens, we constructed a relative POS-tag matrix. Given an input sequence and a dictionary containing k POStags, the algorithm applies a formular within the embedding logits to compute the relative matrix entries. The resulting values are then mapped through an embedding layer, ensuring a unique and consistent encoding of the correlation between the POS tags of each token pair.

$$P_{[m\times m]} : n_{[i,j]=Embedding(pos_i \times k + pos_j)}$$

Relative Positional Encoding: As introduced by [22] and later adopted in modern models such as the Text-to-Text Transfer Transformer (T5), this approach learns relative positional information based on the distance j−i, where i denotes the query position and j the key position. By directly modeling token-to-token distances within a sequence, the method captures the relative importance of tokens with respect to one another.

For an input sequence of length n, relative positional embedding first constructs an n×n matrix, where each entry $a_(i, j)$ = j−i. The distance values are then shifted from the range [−max_distance, max_distance] to [0, 2 × max_distance] to enable indexing into the embedding table.

Combining these layers' result, the combined attention logit can be written in the form of:

$$Attention^{(h)} = softmax(\frac{QK^T}{\sqrt{d_k}} + B^{(h)} + M_L + P)V$$

Where

- L: The sequence length
- Q: Query matrix (current token)
- K: Key matrix (each other token representation in the sentence)
- V: Value matrix (value of each other token)
- d_k: head_dim (equal to hidden_dim / n_head)
- $B^{(h)}$: Relative Positional Encoding
- M_L: Lexicon Mask
- P: Relative POStag encoding

PhoBERT Dual-End Integration: We firstly used the pretrained **PhoBERT-base** model [25] to generate the text embedding as input for the Encoder and later applied a Linear combination of this PhoBERT general embedding and the Encoder state with a learnable parameter α, this is considered as our custom **Skip Connection** feature.

$$h''_t = \alpha h_t + (1 - \alpha) h'_t$$

Where:

- h_t: Output embedding from Encoder
- h'_t: Output embedding from PhoBERT (raw embedded Input)
- h''_t: Aggregated output embedding for the Decoder

The **Decoder** architecture resembles the similar structure from [17] but was modified by removing the initial multi-head self-attention layer and adding a custom DecoderHead to better align with our experimental prediction target. The DecoderHead consists of a two-layer MLP followed by a Linear layer that maps to the respective output dimensions for Self-root, Relation, and Second-Entity span indices. We further designed a recursive prediction head, where the output from the previous prediction is concatenated with the decoder's hidden state and used as input for predicting the subsequent target for the scope of experimenting the features' interrelation.

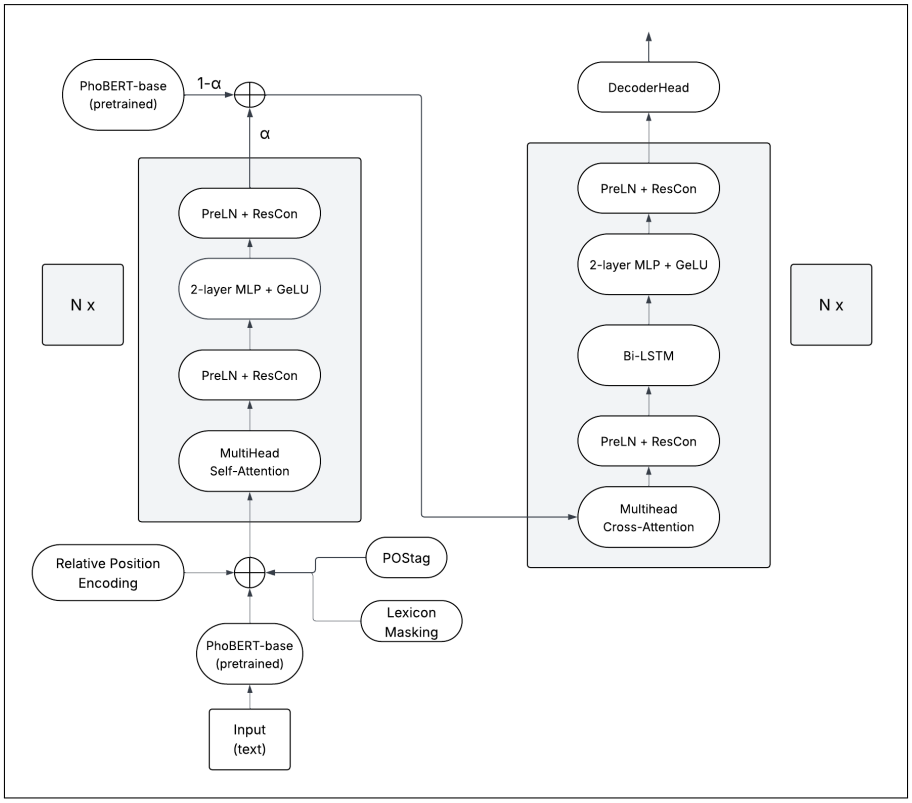


Figure 7: Document Relation Extraction Model

Due to the high complexity of the task, we shifted our focus toward optimizing the selection of layers and the number of encoder–decoder stacks. We began with a 1 Encoder – 1 Decoder architecture as the baseline and experimented with adding a POS-tag layer and a recursive prediction head (Self-root → Relation → Second Entity). Although these additions slightly improved evaluation metrics, they introduced significant computational overhead.

To balance performance and efficiency, we removed these extra layers and instead increased the number of decoder stacks to complement the 12 encoder layers already present in the PhoBERT encoder. We also replaced Relative Positional Encoding with Sinusoidal Positional Encoding, this configuration resulted in the best performance for Self-root and Relation prediction, with only a minor trade-off in the Precision/Recall of second-entity span extraction.

To evaluate the proposed configurations, we employed standard classification metrics, including Accuracy, Precision, and Recall, for each prediction head. For the **"Relation"** classification task, we additionally report Macro-F1, which computes the F1-score independently for each class and then averages across all classes. This metric is particularly suitable for handling class imbalance, as it assigns equal importance to all labels regardless of their frequency in the dataset. For the **"second-entity span"** prediction, a prediction is considered a true positive only if both the start and end indices exactly match the corresponding ground-truth span, ensuring a strict and precise evaluation of span extraction performance. The initial Loss function assigned the same weight to the 3 heads.

$$L_{original} = l_{root} + l_{relation} + 0.5 \times (l_{start_idx} + l_{end_idx})$$

Approach	Self-root			Relation		2nd entity span	
	Accuracy	Recall	Precision	Accuracy	Macro-F1	Precision	Recall
1 Encoder - 1 Decoder (base Head + RelPE)	0.995	0.978	0.993	0.953	0.750	0.620	0.620
1 Encoder - 1 Decoder + POStag	0.994	0.974	0.992	0.957	0.798	0.795	0.795
1 Encoder - 1 Decoder + POStag + RecursiveHead	0.9951	0.995	0.976	0.96	0.819	0.712	0.712
1 Encoder - 3 Decoder	0.995	0.9965	0.975	0.961	0.81	0.939	0.939
1 Encoder - 3 Decoder + SinPE	0.9983	0.9964	0.9938	0.9734	0.8851	0.907	0.907
3 Encoder - 3 Decoder	0.9965	0.9965	0.983	0.964	0.825	0.913	0.913

Table 7: Performance comparison across different approaches

After considering the best model architecture, we retrained the model one more time by reframing the Loss function as:

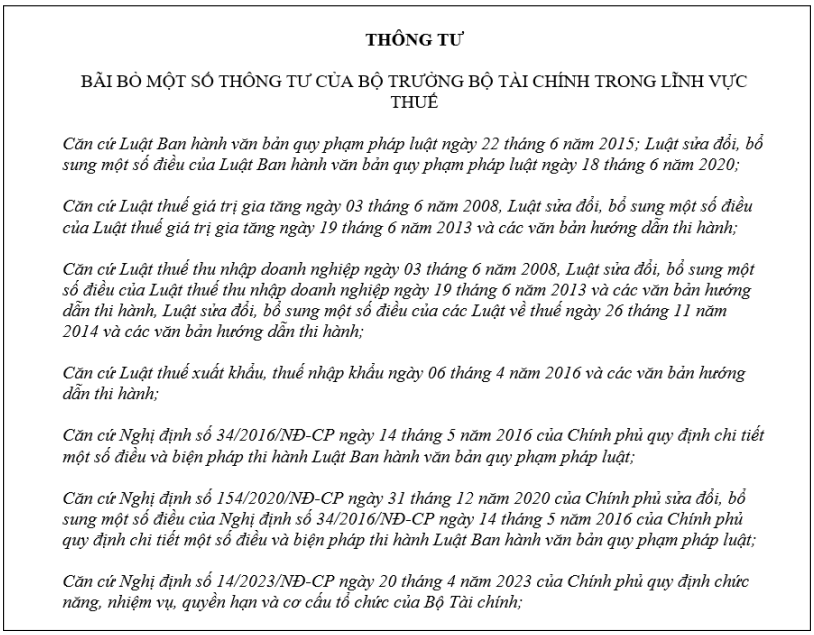
$$L_{new} = 0.5 \times (l_{root} + l_{relation}) + l_{start_idx} + l_{end_idx}$$

focusing further on improving second-entity span prediction, we also unfroze the PhoBERT encoder to enable full end-to-end training of the model backbone, allowing the model to learn task-specific representations and achieve higher performance during deployment.

With the result from the RE model, we further defined three different post-processing approaches to deal with each Relation type proposed above.

Document Cross-Referencing

Document cross-references typically appear at the beginning of a document, positioned between the document title and the start of the main text, and are separated by semicolons (“;”). Based on this observation, we assumed that relations extracted from the **first** sentence of the document correspond to document-level cross-references and yield ”Self-root = 1” in the RE prediction. We then retrieved these relation triplets, extracted the relative metadata of each target document and set up an edge connecting it to the current document node.



THÔNG TƯ

BÃI BỎ MỘT SỐ THÔNG TƯ CỦA BỘ TRƯỞNG BỘ TÀI CHÍNH TRONG LĨNH VỰC THUẾ

Căn cứ Luật Ban hành văn bản quy phạm pháp luật ngày 22 tháng 6 năm 2015; Luật sửa đổi, bổ sung một số điều của Luật Ban hành văn bản quy phạm pháp luật ngày 18 tháng 6 năm 2020;

Căn cứ Luật thuế giá trị gia tăng ngày 03 tháng 6 năm 2008, Luật sửa đổi, bổ sung một số điều của Luật thuế giá trị gia tăng ngày 19 tháng 6 năm 2013 và các văn bản hướng dẫn thi hành;

Căn cứ Luật thuế thu nhập doanh nghiệp ngày 03 tháng 6 năm 2008, Luật sửa đổi, bổ sung một số điều của Luật thuế thu nhập doanh nghiệp ngày 19 tháng 6 năm 2013 và các văn bản hướng dẫn thi hành, Luật sửa đổi, bổ sung một số điều của các Luật về thuế ngày 26 tháng 11 năm 2014 và các văn bản hướng dẫn thi hành;

Căn cứ Luật thuế xuất khẩu, thuế nhập khẩu ngày 06 tháng 4 năm 2016 và các văn bản hướng dẫn thi hành;

Căn cứ Nghị định số 34/2016/NĐ-CP ngày 14 tháng 5 năm 2016 của Chính phủ quy định chi tiết một số điều và biện pháp thi hành Luật Ban hành văn bản quy phạm pháp luật;

Căn cứ Nghị định số 154/2020/NĐ-CP ngày 31 tháng 12 năm 2020 của Chính phủ sửa đổi, bổ sung một số điều của Nghị định số 34/2016/NĐ-CP ngày 14 tháng 5 năm 2016 của Chính phủ quy định chi tiết một số điều và biện pháp thi hành Luật Ban hành văn bản quy phạm pháp luật;

Căn cứ Nghị định số 14/2023/NĐ-CP ngày 20 tháng 4 năm 2023 của Chính phủ quy định chức năng, nhiệm vụ, quyền hạn và cơ cấu tổ chức của Bộ Tài chính;

Figure 8: Document Cross-References appear at the start of the document

As our relation extraction model still exhibits certain performance limitations in predicting second-entity spans, we enhanced this component by integrating the previously trained NER model to recognize target Title, Date, and Document ID. This augmentation improves the reliability of entity resolution and

facilitates later indexing steps by exactly identifying the document ID.

However, our approach faced the problem of Document ID lackage, which means if the Document ID is not in the sentence, the Graph processing will automatically ignore that document. This is due to the fact that several documents can have the same Title (same document with different versions) or the same Date (2 documents issued the same day), we explicitly applied a filter to only save the document if the ID appears in the target document.

Table 8: Extracted Cross-References

Text	Self Root	Relation	Span	Issue Date	Title	Document ID
Căn cứ Luật Ban hành văn bản quy phạm pháp luật…	1	Pursuant	Luật Ban hành văn bản quy phạm pháp luật ngày…	22/06/2015	Luật Ban Hành Văn Bản Quy Phạm	–
Căn cứ Luật thuế giá trị gia tăng ngày 03 tháng…	1	Pursuant	Luật thuế giá trị gia tăng ngày 03 tháng 6 năm…	03/06/2008	Luật Thuế Giá Trị Gia Tăng Ngày 03	–
Căn cứ Luật thuế thu nhập doanh nghiệp ngày 03…	1	Pursuant	Luật thuế thu nhập doanh nghiệp ngày 03 tháng…	03/06/2008	Luật Thuế Thu Nhập Doanh Nghiệp Ngày 03	–
Căn cứ Luật thuế xuất khẩu, thuế nhập khẩu ngày 06…	1	Pursuant	Luật thuế xuất khẩu, thuế nhập khẩu ngày 06…	06/04/2016	Luật Thuế Xuất Khẩu, Thuế Nhập Khẩu Ngày 06	–
Căn cứ Nghị định số 34/2016/NĐ-CP ngày 14 tháng…	1	Pursuant	Nghị định số 34/2016/NĐ-CP ngày 14 tháng 5…	14/05/2016	Nghị Định Số 34/2016/NĐ-CP Ngày 14 Tháng 5	34/2016/NĐ-CP
Căn cứ Nghị định số 154/2020/NĐ-CP ngày 31 tháng…	1	Pursuant	Nghị định số 154/2020/NĐ-CP ngày 31…	31/12/2020	Nghị Định Số 154/2020/NĐ-CP Ngày 31 Tháng 12	154/2020/NĐ-CP
Căn cứ Nghị định số 14/2023/NĐ-CP ngày 20 tháng…	1	Pursuant	Nghị định số 14/2023/NĐ-CP ngày 20…	20/04/2023	Nghị Định Số 14/2023/NĐ-CP Ngày 20 Tháng 4	14/2023/NĐ-CP

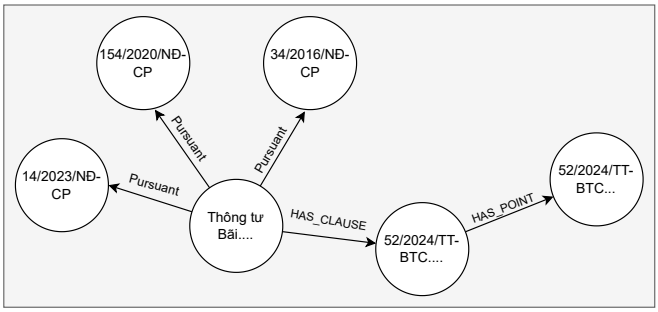


Figure 9: Resulting Cross-Document References, note that only 3 connections are set up due to the recognition of Document ID

Document Node-Referencing

The Node-reference appears when the current Node of the document references 1 or more nodes from another document with or without Document ID.

For example: the sentence ”***Bãi bỏ các Điều 32, 33 và 139, điểm b khoản 1 Điều 163, khoản 3 và khoản 4 Điều 179 của Luật thi hành án dân sự số 26/2008/QH12.***” references 6 other single entities from the Document ID 26/2008/QH12, namely

- ”Điều 32 Luật số 26/2008/QH12”
- ”Điều 33 Luật số 26/2008/QH12”

- ”Điều 139 Luật số 26/2008/QH12”
- ”điểm b khoản 1 Điều 163 Luật số 26/2008/QH12”
- ”khoản 3 Điều 179 Luật số 26/2008/QH12”
- ”khoản 4 Điều 179 Luật số 26/2008/QH12”

which by simply extracting the whole combination is not sufficient to effectively identifying each individual node. To address this challenge, we first defined a parsing function that maps each individual structural text segment to a corresponding Node ID, this was initially accomplished by the structural parsing function. The parsing function requires the input text to follow the **Higher-down rule**, which means if a lower node level is present, its higher level must also exist inside the text.

Text	Node ID	Note
Điểm a.1) khoản 2 Điều 3 Chương 4 Luật Doanh nghiệp số AB/CD/QH	AB/CD/QH_chapter_IV_C_3_P_2_SP_a_SSP_a.1	Sufficient Structure
Điểm b) khoản 1 Điều 1 Nghị định số 23/2020/NĐ-CP	23/2020/NĐ-CP_C_I_P_1_SP_b	Sufficient Structure
khoản 2 Nghị định số 23/2020/NĐ-CP	23/2020/NĐ-CP_P_2	Insufficient Structure (this is wrong)
Điểm a) khoản 1 Luật Lao động năm 2020		Lack of Document ID

Table 9: Node Indexing Condition

In most cases, an entity resides within a ”Sufficient” condition, while the ”Insufficient” structure appears only occasionally, primarily in amending documents that follow different rules for hierarchical splitting. However, this represents a minority of cases and can be temporarily ignored, as we planned to adopt a separate approach for processing the text of amending documents.

We subsequently developed a function to extract individual legal entities from an input sentence. This function operates by identifying higher–lower structural node levels. For example, in the text

“***Điều 32, 33, 34, 51, 138 và 139, điểm b.1.1) khoản 1 Điều 163, khoản 3 và khoản 4 Điều 179 của Luật thi hành án dân sự số 26/2008/QH12***”

nodes at the same hierarchical level are separated by specific delimiters such as commas, “*và*”, or “*hoặc*”, while nodes at different levels are connected solely by whitespace. By combining this observation with regex patterns for each structural type, we utilize a dynamic dictionary to track higher–lower levels based on their relative positions within the sentence and across splitting characters. We also dealt with special cases like ”*Điều 2 đến Điều 5*” or ”*điểm a đến điểm e*” where the sentence only lists the start and end points instead of each individual one and later applied these result with the Indexing function defined above to retrieve each entity’s raw form, Node ID and Node Type. Our example result is illustrated as below, note that the Vietnamese words have all been de-accented to ensure the Compiler’s font consistency, we keep all of the accents in the real tests.

```
Input:
diem a den diem c khoan 2 dieu 32, diem b.1.1) hoac diem e.2) khoan 2 dieu 1 luat so
    20/2019/QH14

Output:
diem a dieu 32 khoan 2 van ban so 20/2019/QH14
(20/2019/QH14_C_32_P_2_SP_a, Subpoint)

diem b dieu 32 khoan 2 van ban so 20/2019/QH14
```

```
(20/2019/QH14_C_32_P_2_SP_b, Subpoint)

diem c dieu 32 khoan 2 van ban so 20/2019/QH14
(20/2019/QH14_C_32_P_2_SP_c, Subpoint)

diem b diem b.1 diem b.1.1 khoan 2 dieu 1 van ban so 20/2019/QH14
(20/2019/QH14_C_1_P_2_SP_b_SSP_b.1_SSSP_b.1.1, Subsubsubpoint)

dieu 1 khoan 2 diem e diem e.2 van ban so 20/2019/QH14
(20/2019/QH14_C_1_P_2_SP_e_SSP_e.2, Subsubpoint)
```

Document Self-Referencing

Document Self-references usually appear from the ”Clause” level down to smaller nodes with a remarkable appearance of ”*X này*” pattern, where X is a Legal structural entity (Document/ Chapter/ Clause/ Point).

For example, the sentence ”**Quyết định của Ủy ban nhân dân phường được ban hành để thực hiện chủ trương quy định tại khoản 1 Điều này và thực hiện chủ trương, biện pháp khác**” has a Self-reference to the node ”Clause 1” of the same document. To address this issue, we optimized the previously defined indexing function by introducing a **Root** parameter, which is passed from the structural parsing step. The Root parameter stores the alternative ID for “X này” references, ensuring consistency in the indexing process.

In more details, when the system processes *“Khoản a”* within *“Điều b”* inside *“Chương c”* of a document, a temporary variable ”chapter_id” holds the value “documentid_chapter_c”, and the temporary variable ”clause_id” stores “documentid_chapter_c_C_b”. If the text of “*Khoản a*” contains a reference such as “**tại Khoản d Điều này**”, the function directly passes the temporary Clause ID “documentid_chapter_c_C_b” as the Root to the indexing step for “*Khoản d + Root*”, returning “**DocumentID_chapter_c_C_b_P_d**” as the target ID.

1. Thuế suất thuế thu nhập doanh nghiệp là 20%, trừ trường hợp quy định tại các khoản 2, 3 và 4 Điều này và đối tượng được ưu đãi về thuế suất quy định tại Điều 13 của Luật này.

2. Thuế suất 15% áp dụng đối với doanh nghiệp có tổng doanh thu năm không quá 03 tỷ đồng.

3. Thuế suất 17% áp dụng đối với doanh nghiệp có tổng doanh thu năm từ trên 03 tỷ đồng đến không quá 50 tỷ đồng.

Doanh thu làm căn cứ xác định doanh nghiệp thuộc đối tượng được áp dụng thuế suất 15% và 17% quy định tại khoản 2, khoản 3 Điều này là tổng doanh thu của kỳ tính thuế thu nhập doanh nghiệp trước liền kề. Việc xác định tổng doanh thu làm căn cứ áp dụng thực hiện theo quy định của Chính phủ.

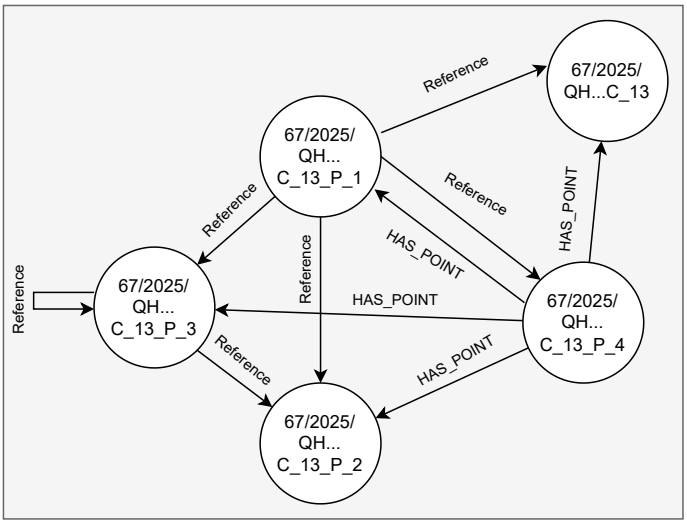


Figure 10: Example Document Self References, as the node "*Khoản 3*" mentions "*Khoản 2, 3 của Điều này*", a Self-loop and an Edge will be constructed

Relational Edge Connection

After the extraction and indexing of the second entities, we utilized the Cypher query language to map these entities into Neo4j GraphDB. Assuming that each of the proposed Node relations stays within one sentence, we set up the Relation extraction pipeline to run through each individual sentence of each node's text as path of the graph structural parsing pipeline. To reduce the computational cost of this practice, we designed a set of filters to be applied to the input sentence, ensuring that:

- The input sentence must contain a Document ID or an "*X này*" combination to ensure the existence of root-document identification in node indexing.
- The Second entity from the RE model must contain at least 1 legal entity from the list [*Luật, Nghị định,...Chương, Điều, khoản,...*].
- The set of multi-entities extracted from a single relation triplet must be deduplicated and checked for structural order consistency as loss of structural hierarchy can appear in several cases.

3.3 QA Benchmark Dataset

3.3.1 Proposed data model

To construct a robust benchmark dataset tailored for our comparison scope, we built a generated QA dataset based on the collected documents corpus. The QA data model was structured to explicitly link each question–answer pair to its source legal document, thereby supporting contextual grounding and validation through rich metadata.

Specifically, each QA entry initially includes:

- **document identifiers (`doc_id`, `doc_source`)** to capture basic information of the document,
- **question and answer** texts grounded in specific legal clause,
- **supporting contexts and corresponding context nodes** to anchor answers to exact legal provisions,

- **generation metadata** such as generation method and timestamps,
- **validation results** including quality scores and reviewer notes.

To support diverse reasoning behaviors, each QA pair is also labeled with:

- **question_type**, capturing the interrogative form (e.g. what, why, how, when);
- **question_category**, capturing the cognitive demand (e.g. factual, explain, compare).

Question Type	Example
What	Theo Điều 3 Luật Giá, Nhà nước định giá một số mặt hàng như thế nào và các quy định pháp luật nào liên quan?
When	Khi nào Sở quản lý ngành tổ chức hiệp thương giá đối với hàng hóa, dịch vụ?
Where	Các tổ chức, cá nhân kinh doanh phải niêm yết giá ở đâu trước khi bán hàng?
Why	Tại sao các hành vi liên quan đến giá bị nghiêm cấm theo quy định pháp luật?
Who	Ai có thể đăng ký hành nghề thẩm định giá và có những điều kiện nào cần đáp ứng?
How	Làm thế nào việc thực hiện bình ổn giá trên phạm vi cả nước khác với phạm vi địa phương?

Table 10: Question Types

Question Category	Description	Example
Factual	Requests legal statements directly defined in the source documents.	Theo Điều 3 Luật Giá, Nhà nước định giá một số mặt hàng như thế nào và các quy định pháp luật nào liên quan?
Clarify	Seeks clarification of legal subjects, roles, or conditions that require interpretation across clauses.	Doanh nghiệp thẩm định giá có bắt buộc phải mua bảo hiểm trách nhiệm nghề nghiệp không?
Enumerate	Requires listing multiple legal requirements, conditions, or criteria specified in the text.	Người đại diện theo pháp luật, Giám đốc hoặc Tổng giám đốc của doanh nghiệp thẩm định giá cần đáp ứng những yêu cầu nào?
Explain	Requires explanatory reasoning, including legal rationale, purpose, or underlying justification of regulations.	Khi nào Sở quản lý ngành tổ chức hiệp thương giá và vì sao cần thực hiện công tác này?
Compare	Involves comparative analysis between statements, scopes, or implementation levels.	Làm thế nào việc thực hiện bình ổn giá trên phạm vi cả nước khác với phạm vi địa phương?

Table 11: Question Categories

3.3.2 Crawled FAQ and Generated dataset

FAQ Crawled Dataset

To construct a grounded and resource-efficient dataset, we use a crawled FAQ dataset as a grounding and enrichment layer for LLM-generated QA data. Several limitations were identified in the crawled official FAQ data, including the absence of explicit supporting context fields and the lack of transparent document source attribution for each question. These issues substantially reduce the reliability and trustworthiness of direct evaluation based on the FAQ alone; as a result, rather than directly reusing FAQ questions, this dataset guides the generation pipeline through typical and semantic constraints. Vietnamese tax law FAQs reflect how taxpayers naturally phrase questions and how authorities communicate complex legal rules in accessible language, helping bridge the gap between formal legal texts and retrieval tasks.

The FAQs are collected through a structured **Extract-Transform-Load (ETL)** crawling pipeline that systematically gathers question-answer pairs from authoritative sources such as government websites (*gdt.gov.vn*) and legal libraries (*thuvienphapluat.vn*). The pipeline uses Selenium-based extractors to handle dynamic content, pagination and JavaScript rendered pages. Each extracted record contains not only the question and answer text but also supporting legal references, timestamps, source URLs and provenance metadata. The data is streamed incrementally into checkpointed CSV files to support reproducibility and fault tolerance.

Field	Value
source	thuvienphapluat
question_source	https://thuvienphapluat.vn/hoi-dap-phap-luat/c...
question	Người nộp thuế có trách nhiệm gì...
answer	Căn cứ theo Điều 17 Luật Quản lý thuế 2019 sửa...
created_time	14:03 | 26/11/2025
metadata	{"main_title": "Công văn 5526/CT-CS 2025 hỗ tr..."}
question_type	What
qa_category	Factual

Table 12: Sample Crawled FAQ

QA Benchmark Dataset Generation

We strategically designed our generated QA framework to consist of several interconnected components.

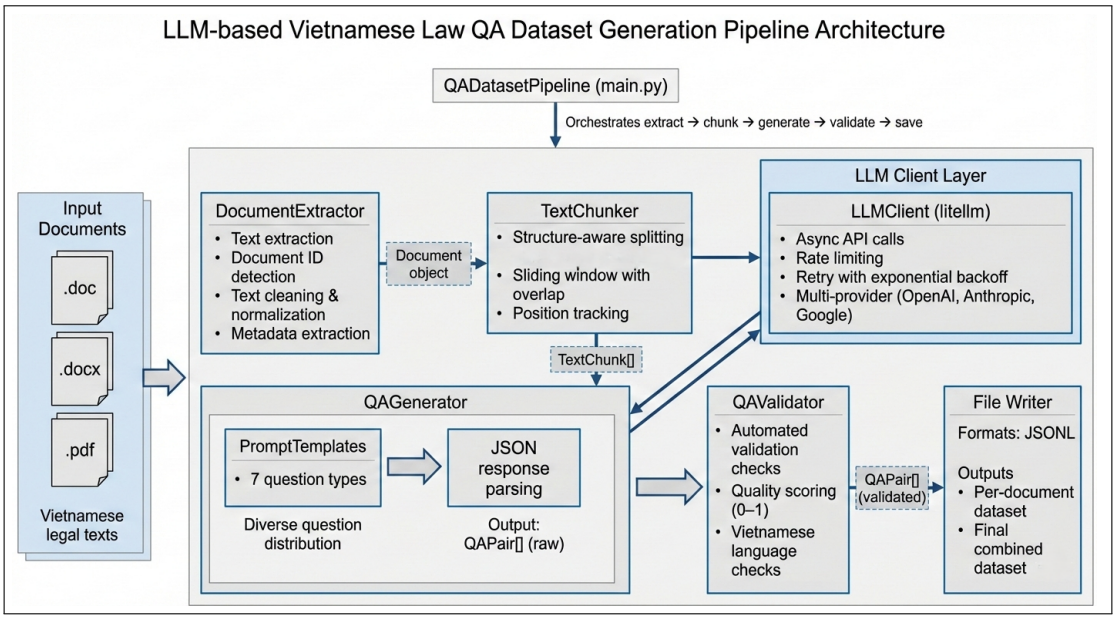


Figure 11: LLM generated QA dataset Data model

DocumentExtractor

The extraction component transforms raw Vietnamese legal documents into clean, structured text for downstream processing. It supports .doc, .docx and .pdf formats. Text is extracted from both paragraphs and tables in the documents as well as page level content in PDF to minimize information loss. Each document is assigned a document ID inferred from Vietnamese legal naming patterns found in filenames or headers. The extracted text is cleaned to remove noise such as page numbers, headers, and control characters while preserving Vietnamese language characteristics.

TextChunker

Chunking splits cleaned text into semantically coherent units aligned with Vietnamese legal structure. Instead of fixed length segmentation, the chunker parses chapters (Chương), articles (Điều), points and subpoints (Điểm) using regular expressions. The smallest meaningful legal units are converted into chunks, each linked to its document ID and assigned a unique chunk ID. Chunks are classified as short, medium or long based on length, ensuring that later stages received context preserving inputs.

LLMClient

The LLM client serves as the communication layer between the pipeline and external LLM APIs. Built on the LiteLLm library, it provides a unified interface for GPT-4 with centralized configuration: default model GPT-4, temperature 0.8, maximum output 2000 tokens, top-p 0.9, and a 60 second timeout. Robustness is ensured through automatic retries (up to 3 attempts with exponential backoff), rate limiting (60 requests/minute) and controlled concurrency (default 5 parallel requests).

QAGenerator

The generation component produces QA pairs from chunked text using a controlled LLM workflow. It prioritizes medium and long chunks to ensure sufficient context, distributes generation evenly across each document and enforces diversity by varying question types and categories. For each target chunk, the generator constructs a prompt that includes the main chunk and 2-5 nearby supporting chunks to invoke the LLM with a slightly higher temperature to encourage variation. Generated results are parsed, checked to ensure Vietnamese language use and stored as standardized QA observations with unique IDs and metadata. This generator takes the input crawled FAQ dataset from the above section as K-shot examples and learns the common questioning patterns to guide the QA generation.

QAValidator

The validator serves as the final quality control stage, filtering and scoring generated QA pairs before adding in the dataset. It mainly applies rule-based checks supplemented by heuristic and similarity based methods. Each validation category produces a normalized score in the range [0,1]. We calculated the final validation score by the mean of all category scores with selected categories allowing partial credit to tolerate minor issues while still penalizing low quality. A QA pair is included in the dataset only if its final score exceeds 0.75 threshold. For traceability, the final score and validation notes are stored with each QA pair to support analysis and future tuning.

Validation Category	Description	Example
Structural adequacy	Checks whether the question and answer have reasonable length and enough information	Question and answer within length limits (10–500 / 10–2000 chars) → **Score: 1.0**
Linguistic quality	Checks grammar, clarity and natural Vietnamese usage	Contains Vietnamese question words and diacritics → **Score: 1.0** English text → **Score: 0.0**
Semantic quality	Checks whether the answer adds meaningful information beyond the question	Answer explains beyond the question → **Score: 1.0** Answer repeats question wording → **Score: 0.0**
Contextual grounding	Verifies that the answer is supported by the provided document context	Strong keyword overlap with context (≥ 0.3) → **Score: 1.0** Weak overlap → **Score: 0.7**
Legal consistency	Ensures the answer correctly reflects the legal meaning of the cited clause	Correctly states "30 ngày" as cited → **Score: 1.0** States "60 ngày" instead → **Score: 0.0**
Safety/Cleanliness	Detects empty, malformed or corrupted content	Properly formatted Q&A → **Score: 1.0** Empty "N/A" / single-word answer → **Score: 0.0**

Table 13: Validation Criteria and Examples

Field	Value
question_id	48/2024/QH15_Q02
doc_id	48/2024/QH15
doc_source	luat_48/2024/QH15.pdf
question	Tại sao giá bán hàng hóa, dịch vụ cần được xác định chưa có thuế giá trị gia tăng?
answer	Giá bán hàng hóa, dịch vụ cần được xác định chưa có thuế giá trị gia tăng để đảm bảo tính đúng đắn và nhất quán trong việc xác định giá trị chịu thuế, đặc biệt khi hàng hóa, dịch vụ có thể chịu các loại thuế khác như thuế tiêu thụ đặc biệt và thuế bảo vệ môi trường.
supporting_context	["a) Đối với hàng hóa, dịch vụ do cơ sở kinh doanh bán ra là giá bán chưa có thuế giá trị gia tăng; đối với hàng hóa, dịch vụ chịu thuế tiêu thụ đặc biệt là giá bán..."]
created_time	2025-12-22 05:04
question_type	Why
question_category	Explain
cognitive	False
multi_clause	True
negative	False
multi_source	False
validation_score	1.0
validation_note	Passed all checks!

Table 14: Final Sample QA Dataset

3.4 Retrieval Pipeline and Graph-Integrated Retrieval

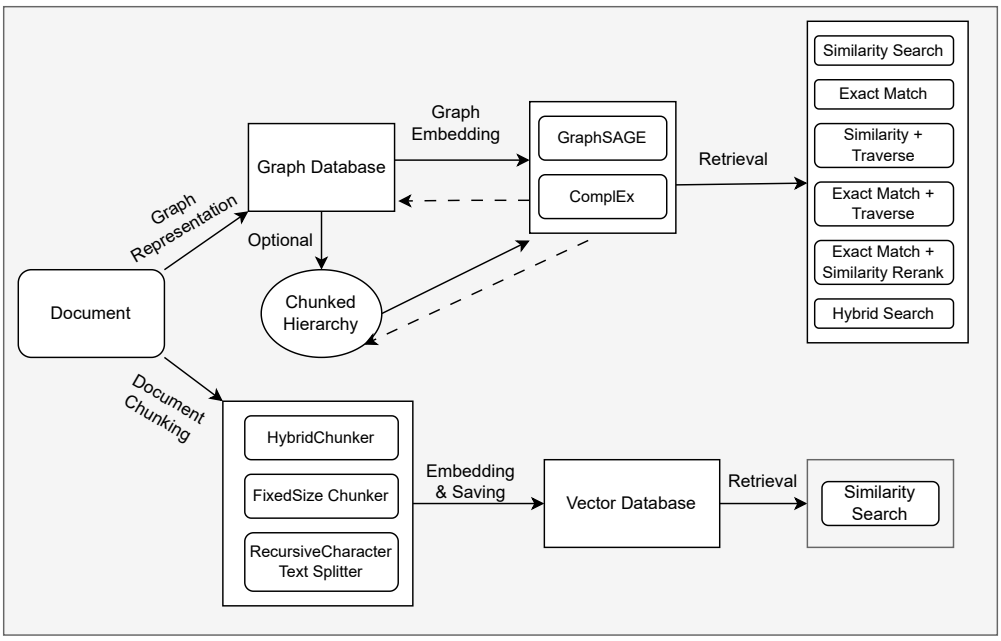


Figure 12: Retrieval pipeline

3.4.1 Vector Retrieval

To establish a robust baseline for evaluating the effectiveness of our proposed Graph-based approaches, we firstly implemented a traditional Vector Retrieval framework. This pipeline focuses solely on the semantic similarity ranking between the input query and document segments, serving as the fundamental control group for our comparative analysis against the Graph-augmented methods. The framework is

composed of three consecutive stages: Document Chunking, Textual Embedding, and Similarity-based Retrieval.

Chunking Strategy

We began the first stage by utilizing several chunking strategies adapted from traditional vector retrieval frameworks. These strategies are applied uniformly across general document structures without explicitly accounting for document-specific formatting. The Docling *HybridChunker* performs structure-aware segmentation by preserving coherent blocks or paragraphs within a single chunk and avoiding mid-paragraph splits, while maintaining a consistent target chunk size. As a result, it produces chunks of varying lengths, each composed of smaller hierarchical components.

In contrast, the *RecursiveCharacterTextSplitter* from the **LangChain** pipeline aims to preserve semantic coherence by recursively splitting text using an ordered list of separators, typically paragraph breaks and newline characters. While this method largely preserves paragraphs and sentences, it focuses primarily on individual escape characters without explicit awareness of hierarchical structure.

In addition to the chunking strategies provided by existing frameworks, we also experimented with a *Fixed-SizeChunking* strategy. In this approach, the document text is treated as a linear sequence of words-characters and segmented into fixed-length blocks of N words (with N=200 in our experiments), without overlap between chunks. This method completely ignores document structure and focuses solely on the textual content, serving as a random baseline for the experiment.

Text Embedding and Vector Database

Following segmentation, the retrieval system relies on embedding models to transform the resulting context chunks into dense vector representations. Model selection prioritized architectures capable of capturing contextual and domain-specific characteristics of Vietnamese legal text. We primarily employed PhoBERT-base, which has been trained on large-scale Vietnamese corpora, including news articles, Wikipedia, and other high-quality public text sources. This training enables PhoBERT to effectively capture both the structural and semantic properties of Vietnamese documents in general, and Vietnamese legal documents in particular. The Byte-Pair Encoding (BPE) mechanism employed by PhoBERT is an additional advantage, as the model is trained on Vietnamese-specific word segmentation, thereby preserving semantically meaningful word units rather than relying solely on general subword splitting.

To provide a comprehensive benchmark, we additionally evaluated several strong multilingual sentence embedding models that support Vietnamese, including *paraphrase-multilingual-MiniLM-L12-v2*, *all-mpnet-base-v2*, or *all-MiniLM-L12-v2* from the SentenceTransformers library. All generated embeddings were then indexed in an in-memory Chroma vector database to facilitate efficient querying. For the retrieval phase, we employed a standard Similarity Search mechanism that calculates the Cosine Similarity between the query embedded vector and the stored chunk vector then returns the top k chunks with the highest corresponding similarity score.

3.4.2 Graph Retrieval

Chunked Hierarchy

To support graph-based retrieval, in addition to the applied hierarchical node splitting defined in the '*Graph Representation*' section, we implemented a post-graph level-wise chunking strategy. For each level, the text was combined into chunks by aggregating the current node with its child components, text embeddings were then computed for each resulting chunk. These chunk was assigned an identifier derived from the original node ID adding the suffix "_chunk" and labeled as "Chunk" to support the chunk-only retrieval mode; moreover, an **IS_IN** relationship was created from the corresponding inner

node to its associated chunk. This approach takes advantage of the graph traversability and allows for dynamic chunk size to match the required context conditions.

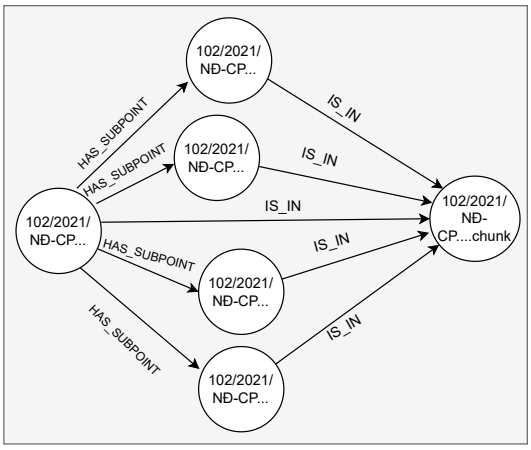


Figure 13: Example of the Chunked Hierarchy method, where a Node and its child components are combined into 1 Chunk node connected by IS_IN relationship

Graph Structural Embedding

Apart from using the raw text embedding from the textual embedder like traditional RAG approaches, we applied two different Graph embedding approaches to fully utilize the graph structure and its rich network of contextual relations.

GraphSAGE

The **GraphSAGE** algorithm, first introduced in 2017 by [35], has become one of the most widely adopted layers in graph machine learning models. It is designed to effectively capture relational information encoded in node neighborhood and to represent such relationships within individual node embeddings. The GraphSAGE algorithm operates as follows:

Given a graph consisting of nodes connected by edges, where each node is associated with a feature representation vector h_x, the algorithm first applies neighborhood aggregation of feature vector, which is by aggregating the current node's feature with its neighbors' features, this aggregation function can either be a Mean, Max Pooling or even a small Long-Short Term Memory (LSTM) layer.

$$h_{N(v)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{h_u^{(k-1)} \mid u \in N(v)\right\}\right)$$

The aggregated hidden representation is then concatenated with the node's original feature matrix and passes through a sigmoid-normalized linear layer to get the output hidden state of the current layer.

$$h_v^{(k)} = \sigma\Big(W^{(k)} \cdot \text{CONCAT}\Big(h_v^{(k-1)},\ h_{N(v)}^{(k)}\Big)\Big)$$

To mitigate the high computational cost of the algorithm and to ensure fault tolerance, we sampled a subset of the full document corpus and evaluated the graph embedding approach in conjunction with the custom chunking strategy described above. For the input data, we directly used textual node embeddings as inputs to the algorithms with the view to enriching these contextual embedding with positional features. To address the absence of textual content at the document's central node, we incorporated two of its attributes—**document_type** and **amend**—thereby utilizing general document characteristics to enrich the graph embedding representation while still preserving the graph consistent structure.

For the GraphSAGE model training, we input directly input three of the node's feature as input, the resulting embedding is the embedded combination of both node's textual and positional representation.

- **original_embedding**: The textual embedding derived from the node's associated text using the embedding model. For nodes that do not contain explicit textual content, such as central nodes, the corresponding embeddings are initialized to zero vectors.
- **document_type**: The document type is encoded using a predefined mapping dictionary and is available only at the central node. For all other nodes, the document_type attribute is set to zero.
- **amend**: Whether a document is an amending document is represented as a binary flag (0/1) at the central node. For all other nodes, this attribute is set to -1.

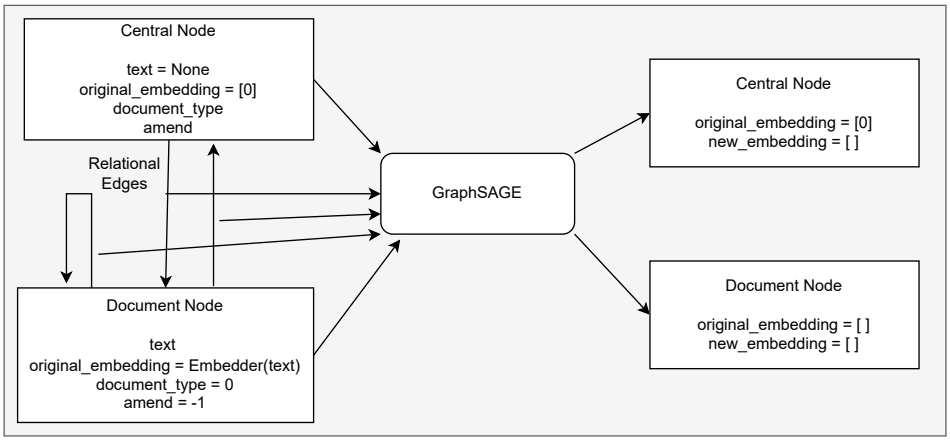


Figure 14: GraphSAGE pipeline

ComplEx

The **ComplEx** embedding algorithm introduced and employed in researches like [29], despite less widely adopted than GraphSAGE or traditional graph embedding methods such as GCN or GIN, has demonstrated effectiveness in encoding graph relational structures into node embedding vectors by operating on triplet data of the form (source, edge_label, target) and projecting each entity and relation into a Complex vector space.

$$e = e^{\text{Re}} + i\, e^{\text{Im}}, \qquad r = r^{\text{Re}} + i\, r^{\text{Im}}$$

For a triplet (s,r,o) representing (source-relation-object) of a node relation, the algorithm applies a complex scoring function utilizing the Tri-linear dot-product utilizing both the Real and the Imaginary components of the representation.

$$f(s,r,o) = \langle s^{\text{Re}}, r^{\text{Re}}, o^{\text{Re}} \rangle + \langle s^{\text{Re}}, r^{\text{Im}}, o^{\text{Im}} \rangle + \langle s^{\text{Im}}, r^{\text{Re}}, o^{\text{Im}} \rangle - \langle s^{\text{Im}}, r^{\text{Im}}, o^{\text{Re}} \rangle$$

Where the tri-linear dot-product can be expressed as:

$$\langle a, b, c \rangle = \sum_{k=1}^{d} a_k b_k c_k$$

The result is then used to calculate the updated model parameter using the learning rate η and negative loss function with negative sampling (denoted by binary flag y).

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial \theta^{(t)}}$$

Where

$$\mathcal{L} = -\sum_{(s,r,o,y)\in\mathcal{D}} \log \sigma\big(y \cdot f(s,r,o)\big)$$

The resulting embedding from ComplEx is a k-dimensional vector in which each dimension is represented as a complex number composed of both real and imaginary components. To maintain the compatibility with standard vector similarity computations, the real and imaginary components of each dimension are flattened, yielding a new vector of dimension 2k.

$$ComplEx = [a + bi\,, c + di\,, e + fi\,, ...] -> [a, c, e, ..., b, d, f, ...]$$

For graph structure embedding using ComplEx, we adopted a different approach from GraphSAGE. We first extracted head–relation–tail triplets from the graph structure and computed the node positional embeddings based on these triplets. The original embeddings and the newly constructed positional embeddings were then fed into a Gated Fusion Layer, which produces a fused embedding of dimension k, matching the dimensionality of the text embedding. The resulting representation integrates textual information and positional patterns into a single, unified embedding with the same original dimension.

The idea of the Gated Fusion Layer was first introduced in [34] and later adopted by recent works such as [33], starting with the initial textual and positional Embeddings

$$\mathbf{t} \in \mathbb{R}^k \quad \text{(text embedding)}, \quad \mathbf{e} \in \mathbb{R}^n \quad \text{(KG embedding)}$$

the algorithm firstly projects the KG embedding into the space by a Linear layer, concatenates the 2 embeddings and computes the gate vector by another sigmoid-normalized Linear layer.

$$e' = \mathbf{W}_p\mathbf{e} + \mathbf{b}_p$$

$$\mathbf{g} = \sigma\big(\mathbf{W}_g[\,\mathbf{t} \parallel e'\,] + \mathbf{b}_g\big)$$

The final representation is obtained by fusing the outputs using a gating function, which produces a new k-dimensional vector that matches the dimensionality of the original text representation.

$$\mathbf{f} = \mathbf{g} \odot \mathbf{t} + (1 - \mathbf{g}) \odot \mathbf{e}'$$

Where $\odot$ is the matrix element-wise multiplication or **Hadamard product**.

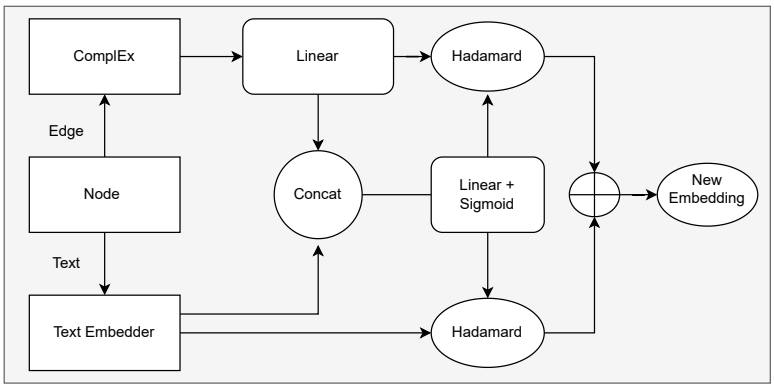


Figure 15: ComplEx Embedding pipeline with Gated Fusion

We finally added all of these Embedding features as the Node attributes, ensuring all Nodes contain

- **original_embedding**: The textual Embedding from the embedding model, in Document central node where no text attribute is given, this embedding is all zero

- **positional_embedding**: The positional Embedding from the ComplEx model training, calculated using all the Relations associated to each individual node.
- **embedding**: The combined Embedding from original_embedding and positional_embedding through the Gated Fusion Layer in the ComplEx approach, or the resulted Positional-embedded textual representation of the text in the GraphSAGE approach
- **node_id**: The id of the node, which will be returned along with the node text for transparent source citations.

Graph Retrieval

For graph retrieval, we first facilitated baseline retrieval strategies based on **embedding similarity** (dense representation) and **exact keyword matching** (sparse representation), as well as a combined approach that performs exact word match followed by a re-ranking step using embedding similarity, following the proposed retrieval technique from [29]. We additionally implemented a hybrid ranker that jointly leverages dense and sparse feature representations. These methods closely resemble traditional retrieval approaches, with the primary distinction being that the embeddings are already enriched with graph node relational information. To align the query embedding with the output space of the gated fusion module, which integrates contextual and graph-structural information by an MLP layer, we introduced two adaptive strategies.

- In the first approach, the textual embedding of the input prompt is fused with a zero vector representing the initial absence of graph structure, yielding a combined representation in the joint embedding space.
- In the second approach, the top-k contexts are first retrieved using exact lexical matching and the positional embeddings from these contexts are mean-pooled. This resulting pooled vector serves as a pseudo-positional embedding that maps the input prompt to a virtual location in the graph based on its lexically-related neighbors, which is then fused with the textual embedding to produce the final representation to be applied for second-turn similarity search.

Beyond basic node retrieval, we utilized the fundamental property of graph, being the **Graph Traversal**. After retrieving an initial set of candidate nodes, we expanded the retrieval scope by incorporating their neighboring nodes within k-hops (i.e., direct neighbors and neighbors of neighbors), subject to a maximum of n nodes. The retrieved nodes are ordered by node ID and concatenated with the original node text to construct the final chunk result. This strategy captures both node-specific information and local neighborhood context within a single contextual chunk.

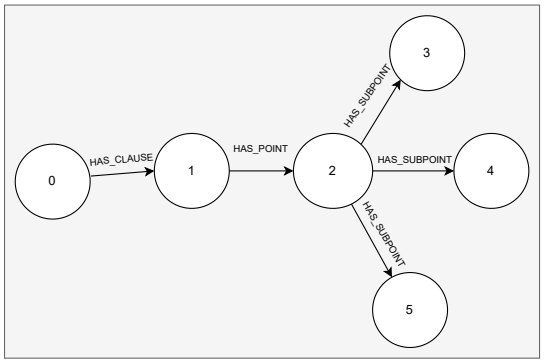


Figure 16: Sample Graph Traversal, the system first retrieves Node 2 then collects the top 5 nodes within 2 hops neighboring nodes (0, 1, 3, 4, 5) ranked by node ID and concatenates all of the text into 1 chunk

3.5 Demo UI Visualization

To facilitate clearer visualization of each phase of the project, we developed a demonstration user interface with use cases aligned to the corresponding stages of the proposed pipeline. This UI main features consist of 4 main use cases:

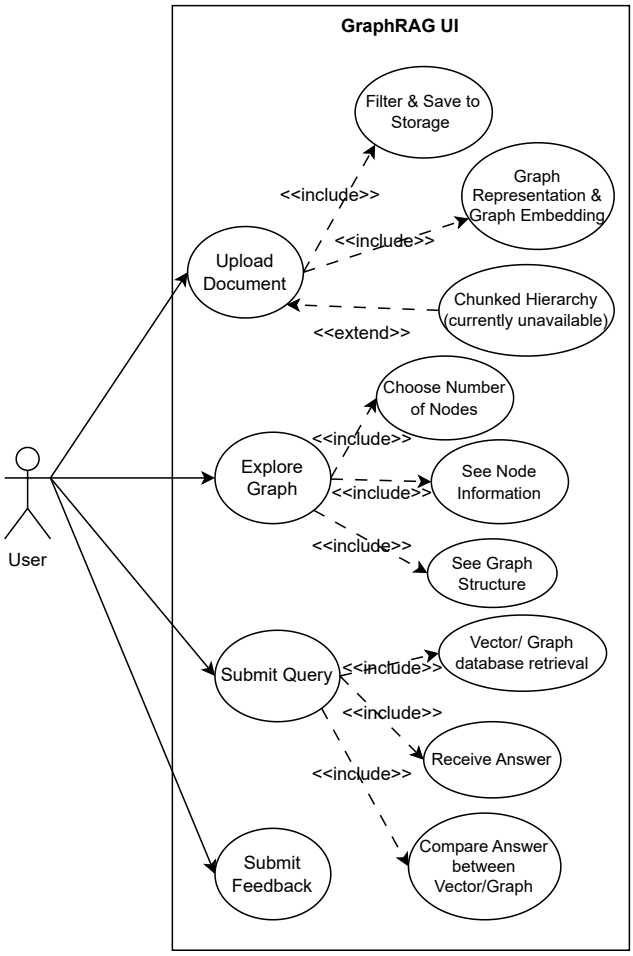


Figure 17: UML Use case Diagram

- **Upload Document**: The first use case focuses on document uploading. The system supports the ingestion of documents in `.doc`, `.docx`, and `.pdf` formats and applies a normalization step that converts all inputs into the pdf format for centralized processing. The system then filters the uploaded documents based on predefined criteria for Vietnamese legal document types and disregards files that do not meet these requirements. Documents that satisfy all criteria are stored in an object storage, after which their full textual content is parsed for graph-based representation.

  The system also supports the Chunked Hierarchy technique and Graph embedding for enhancing Graph-integrated context retrieval; however, these features will operate very slowly and are currently only used in experiments.

- **Explore Graph**: After transforming the document into a graph-based representation, the system visualizes the resulting graph as an interactive interface. This interface enables users to inspect the

overall graph structure and navigate to individual nodes to view their associated attributes along with choosing the number of represented nodes.

- **Submit Query**: The user interface supports query execution over both vector databases and graph databases. To limit configuration complexity, the retrieval modes are restricted to similarity-based search for the vector database and hybrid search for the graph database. The system additionally supports parallel retrieval, enabling direct comparison between vector-based and graph-integrated retrieval during answer generation.
- **Feedback Submission**: In addition to receiving generated answers, users are provided with the ability to submit feedback for each individual response through both multiple-choice selections and free-text notes. All feedbacks are stored in a centralized repository to support future system refinement and evaluation.

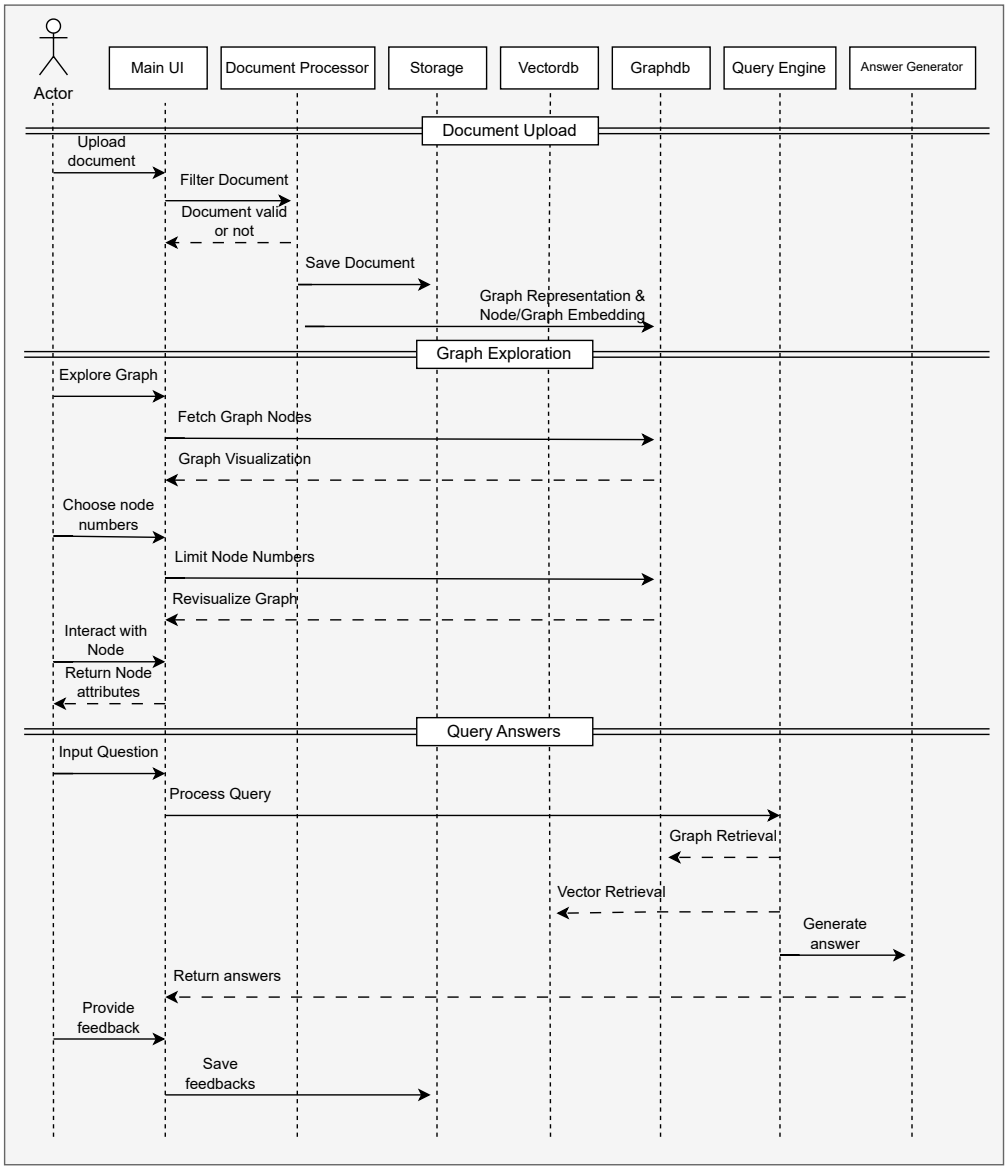


Figure 18: UI Sequence Diagram

4 Results & Discussion

4.1 Evaluation Metrics

To assess the performance of the Retrieval approaches, we defined the set of three benchmark evaluation metrics from several RAG studies [38] [39], namely **Precision**, **Recall**, and **Mean Reciprocal Rank (MRR)**. Recent research about RAG optimization has established these metrics as the standard for evaluating retrieval performance; however, our project utilizes these metrics primarily as a shared baseline for assessing the performance of our approaches instead of directly comparing with other researches due to scale mismatches.

Retrieval evaluation does not involve a negative class; instead, it focuses on whether an item is *relevant* to the golden contexts. Given a list of contexts retrieved by the retrieval model and a corresponding list of referenced contexts representing the ideal (ground-truth) set, the evaluation metrics are defined as follows:

$$Recall = \frac{|Retrieved_Context \cap Referenced_Context|}{Referenced_Context}$$

$$Precision = \frac{|Retrieved_Context \cap Referenced_Context|}{Retrieved_Context}$$

Recall measures the proportion of referenced contexts covered and supported by retrieved information. **Precision**, in contrast, measures the proportion of retrieved contexts that directly support the gold referenced contexts, penalizing the retrieval of irrelevant or unsupporting contexts. These metrics operate in a complementary trade-off, particularly in greedy retrieval settings where increasing the number of top-k retrieved contexts generally leads to higher recall, as more relevant contexts are likely to be included; however, this increase comes at the cost of reduced precision due to the introduction of a larger number of irrelevant contexts.

A distinct aspect of these metrics is the inter-entity cross-comparison between two lists, where the notation $\cap$ denotes **Relevance** between entities from each list. Recent studies have proposed various methods to approach this notion of relevance. For instance, some GraphRAG-based works, such as [38], construct graph-supported benchmark datasets with consistent context chunk identifiers, enabling direct comparison between retrieved and reference identifier lists. In contrast, other studies leverage large language models (LLMs) like the recently introduced **Retrieval Augmented Generation Assessment** (RAGAS) library to infer relevance relationships using richer semantic features, particularly in scenarios where explicit node or chunk identifiers are unavailable. We adopted a thresholded similarity-score approach inspired by RAGAS to support both vector-based retrieval (text-based, stateless chunking) and graph-based retrieval (hierarchical structure with explicit node identifiers). Specifically, a shared embedding model was used to encode both retrieved contexts and reference contexts into a shared embedding space. Pairwise similarity scores were then computed, and a relevance flag was assigned when the similarity score exceeds a predefined threshold, in this study, we benchmarked the PhoBERT-base model as the judge embedding model and adopted a similarity threshold of 0.7 to capture a broad range of result patterns while maintaining a high baseline for Relevancy.

In addition to Recall and Precision, we also evaluated **Mean Reciprocal Rank (MRR)**, which exploits the ranked nature of retrieval systems by assigning higher scores when relevant contexts are retrieved at earlier ranks.

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Where i is the Rank of the first *Relevant* retrieved context to any referenced context.

4.2 Results

We first evaluated a traditional vector-based retrieval approach using embedding similarity over a vector database constructed with three different chunking strategies, which served as the baseline for comparison. We then conducted experiments using our proposed hierarchical chunking technique, along with the associated graph traversal and specialized chunking methods. Subsequently, we evaluated the integration of graph embeddings using both ComplEx and GraphSAGE, under three retrieval paradigms: (i) similarity-based search, (ii) hybrid search combining exact lexical matching and embedding similarity for ranking, and (iii) exact-match retrieval followed by re-ranking using embedding similarity.

For context retrieval using similarity scores over graph-positional embeddings, we initially evaluated three query processing strategies within the same retrieval framework: directly comparing the input prompt embedding with node embeddings, and two fusion-based approaches described above. Empirically, direct similarity-based retrieval achieved the best overall performance. Consequently, we adopted this approach in our final system to simplify the pipeline and reduce the number of experimental trials.

Table 15: Evaluation Metrics for Vector and Graph Configurations

Approach	Method	Chunking	Embedding	Precision	Recall	MRR
Vector	Similarity Search	HybridChunker	Node	0.89	0.91	0.90
Vector	Similarity Search	RecursiveCharTextSplitter	Node	0.89	0.90	0.90
Vector	Similarity Search	Fixed-size Chunking	Node	0.68	0.69	0.68
Graph / Vector	Similarity Search	Hierarchical	Node	**0.91**	**0.93**	**0.92**
Graph	Similarity Search + Graph Traversal	Hierarchical	Node	0.91	0.93	0.92
Graph	Similarity Search	Chunked Hierarchy	Node	0.91	0.93	0.92
Graph	Similarity Search	Hierarchical	ComplEx	0.85	0.90	0.89
Graph	Hybrid Search	Hierarchical	ComplEx	0.56	0.90	0.70
Graph	Exact Match + Similarity Rerank	Hierarchical	ComplEx	0.87	0.92	0.91
Graph	Similarity Search	Hierarchical	GraphSAGE	0.65	0.87	0.73
Graph	Hybrid Search	Hierarchical	GraphSAGE	**0.89**	**0.98**	**0.97**
Graph	Exact Match + Similarity Rerank	Hierarchical	GraphSAGE	0.84	0.95	0.87

4.3 Discussion

Compared with traditional vector-based retrieval approaches, our proposed chunking strategy achieves a modest yet consistent improvement across all evaluation metrics, all surpassing the 0.9 performance threshold at a similarity score of 0.7. However, the utilization of graph structural embeddings yields different effects across the applied algorithms.

While the resulting positional embeddings from **GraphSAGE** exhibit relatively low performance when used solely with embedding-based similarity search, their effectiveness increases substantially when combined with textual matching. This hybrid retrieval setting leads to a sharp improvement in Recall and MRR compared to similarity search using only pure node embeddings, at the cost of only a minor reduction of 0.2 in Precision. The **ComplEx** model, when applied within a lexicon–embedding combined retrieval setting, yields lower performance metrics compared to the pure node embedding baseline, especially where a sharp drop appears in Hybrid search Precision.

Moreover, extending the use of graph structure through graph traversal and chunked hierarchy produces results similar to those obtained using embedding-based search alone, indicating no actual im-

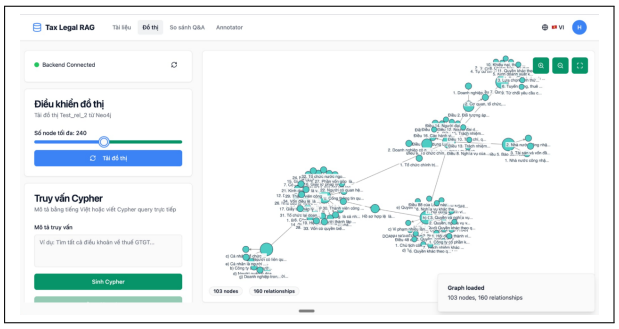
provement in the retrieval performance.

These results can be explained by several factors. First, our approach excludes the use of computationally expensive LLM and instead relies primarily on **HuggingFace** models for textual embeddings. Consequently, model performance is highly sensitive to the choice of embedding model and similarity threshold, which limits the evaluation metrics to being primarily comparative across configurations rather than indicative of standalone research performance. Second, the benchmark dataset does not forcefully require the utilization of graph structure, as it restricts each question to 1 or 2 single context chunks. As a result, graph embeddings intended to model complex cross-node relationships become largely redundant and may even introduce additional noise into the node representations. Moreover, the effectiveness of the graph embeddings themselves may be affected by insufficient training epochs or suboptimal hyperparameter settings, which can substantially influence the quality of the trained vector representations.

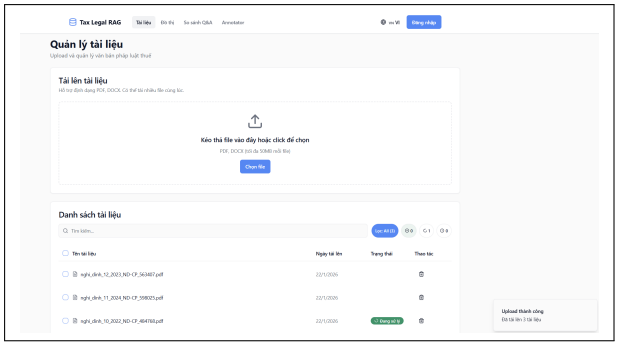
Beyond the limitations related to the experimental results, several opportunities for further optimization remain in the graph representation pipeline. The current processing framework is designed to handle only specific document types such as Laws, Decrees, Resolutions, Circulars, and Decisions due to format inconsistencies. Consequently, other types of legal documents may be rejected or only partially processed, resulting in the loss of substantial content because of format mismatches. Some documents exhibit a multi-document structure, in which a single file contains multiple distinct sub-documents, each with its own title and organizational format. Under the current approach, only the first sub-document is reliably processed, while subsequent sub-documents are ignored. This limitation leads to the omission of potentially significant information and highlights the need for more robust document segmentation and parsing strategies in the future.

Additionally, the raw text–paragraph parsing component does not currently support tables or non-textual elements such as figures and diagrams embedded within documents. As a result, important structural and semantic information contained in these elements is omitted from the graph representation. The core relation extraction process currently operates only when the target entity contains a document identifier or an explicit date, thereby ignoring references that specify only the document title. This limitation arises because node connectivity depends on indexed node identifiers; without a document ID, the algorithm cannot reliably resolve the referenced document. Addressing title-only references would likely require the integration of a larger prebuilt database that maps document titles to their corresponding identifiers to enable accurate entity resolution. Additional challenges in the relation extraction process arise when handling amending documents, which often contain numerous cross-references within each individual sentence following a fixed and formalized structure. These issues are closely related to the node versioning mechanism, which is not yet fully supported in the current system and will be introduced in future work.

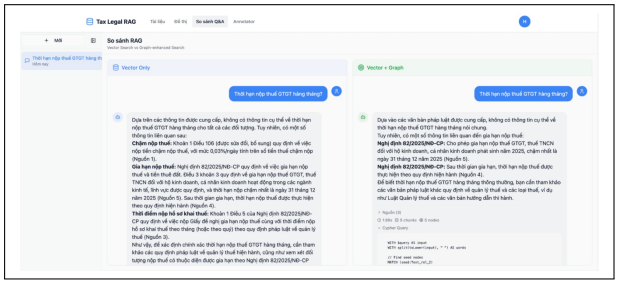
5 UI Visualization



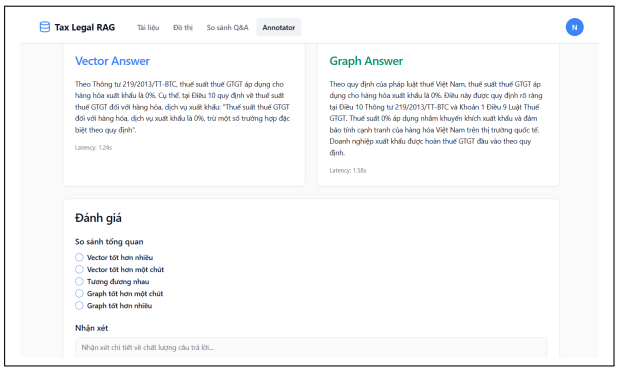
(a) Graph Visualization



(b) Document upload



(c) Answer comparison



(d) QA annotation

Figure 19: User interface overview

6 Conclusion & Future Work

In this project, we have proposed a comprehensive end-to-end pipeline for graph-integrated context retrieval over a corpus of heterogeneous Vietnamese legal documents. The pipeline comprises a document collection framework, a graph construction framework that represents the collected documents as graphs with rich node attributes and explicit cross-references derived from textual citations, and a benchmark question–answer dataset generation framework that is subsequently used to evaluate multiple retrieval strategies leveraging graph structure. Although the experimental results do not fully align with our initial hypotheses, the proposed framework establishes a solid foundation and reveals several promising directions for future improvement and extension.

For future work, we plan to divide our efforts into two main directions: document graph representation and graph-integrated context retrieval. In the document graph representation track, we will further optimize the document processing pipeline to adaptively parse a wider range of document structures beyond those currently supported. Both the document metadata NER model and the relation extraction model will be retrained to more accurately identify relevant information across more diverse and complex cases. In addition, we will complete the integration of the node versioning mechanism to better accommodate the large number of Amending documents and to more accurately reflect document evolution, including the handling of modified or deprecated passage nodes.

Relative to the current humble results in retrieval performance, future works will explore more advanced and expressive retrieval techniques inspired by recent studies. In parallel, the graph embedding component will be further developed through systematic hyperparameter tuning and the research of additional graph representation methods.

References

[1] Chen, Y., Sun, Y., Yang, Z., & Lin, H. (2020). *Joint entity and relation extraction for legal documents with legal feature enhancement.* In Proceedings of the 28th International Conference on Computational Linguistics (pp. 1561–1571).

[2] Nguyen, T. T. T., Nguyen, N. D., Ngo, X. B., & Tu, M. P. (2023). *Joint reference and relation extraction from legal documents with enhanced decoder input.* Cybernetics and Information Technologies, 23(2).

[3] Vu, T., Nguyen, D. Q., Nguyen, D. Q., Dras, M., & Johnson, M. (2018). *VnCoreNLP: A Vietnamese natural language processing toolkit.* In Proceedings of the NAACL-HLT 2018 Demonstrations (pp. 56–60).

[4] Zeng, D., Zhang, H., & Liu, Q. (2019). *CopyMTL: Copy mechanism for joint extraction of entities and relations with multi-task learning.* arXiv preprint arXiv:1911.10438.

[5] Ngo, H. Q., Nguyen, H. D., & Le-Khac, N.-A. (2024). *Ontology knowledge map approach towards building linked data for Vietnamese legal applications*. Vietnam Journal of Computer Science, 11(02), 323–342. https://doi.org/10.1142/S2196888824500015

[6] Zeng, D., Zeng, D., He, S., Liu, K., & Zhao, J. (2018). *Extracting relational facts by an end-to-end neural model with copy mechanism.* In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (pp. 506–514).

[7] Miwa, M., & Bansal, M. (2016). *End-to-end relation extraction using LSTMs on sequences and tree structures.* arXiv preprint arXiv:1601.00770.

[8] Ryu, C., Lee, S., Pang, S., Choi, C., Choi, H., Min, M., & Sohn, J.-Y. (2023). *Retrieval-based Evaluation for LLMs: A Case Study in Korean Legal QA*. In *Proceedings of the Natural Legal Language Processing Workshop 2023* (pp. 132–137). Singapore: Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.nllp-1.13

[9] Vishnuprabha, V., Viswanathan, D. M., Rajesh, R., & Pillai, A. V. (2025). *Augmented Question-guided Retrieval (AQgR) of Indian case law with LLM, RAG, and structured summaries*. arXiv preprint arXiv:2508.04710.

[10] de Oliveira Lima, J. A. (2025). *Poly-vector retrieval: Reference and content embeddings for legal documents*.

[11] Mukund, S. A., & Easwarakumar, K. S. (2025). *Optimizing legal text summarization through dynamic retrieval-augmented generation and domain-specific adaptation*.

[12] Nguyen Ba, T., Doan The, V., Pham Quang, T., & Tran Van, T. (2024). *Vietnamese legal information retrieval in question-answering system*. arXiv preprint arXiv:2409.13699.

[13] Reuter, M., Lingenberg, T., Liepinā, R., Lagioia, F., Lippi, M., Sartor, G., Passerini, A., & Sayin, B. (2025). *Towards reliable retrieval in RAG systems for large legal datasets*. In *Proceedings of the Natural Legal Language Processing Workshop 2025* (pp. 17–30). Association for Computational Linguistics.

[14] de Oliveira Lima, J. A. (2025). *Poly-Vector Retrieval: Reference and Content Embeddings for Legal Documents*. arXiv preprint arXiv:2504.10508.

[15] Huang, Z., Xu, W., & Yu, K. (2015). *Bidirectional LSTM-CRF models for sequence tagging.* arXiv preprint arXiv:1508.01991.

[16] Xu, Y., Mou, L., Li, G., Chen, Y., Peng, H., & Jin, Z. (2015). *Classifying relations via long short-term memory networks along shortest dependency paths.* In Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (pp. 1785–1794).

[17] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need.* arXiv preprint arXiv:1706.03762.

[18] Ha, M. L., Nguyen, T. M. H., Vu, X. L., Nguyen, T. L., Phan, T. H., & Le, V. C. (2020). *VLSP 2020 shared task: Universal dependency parsing for Vietnamese.* In Proceedings of the VLSP 2020 Workshop.

[19] Tran, D. X., Truong, T. D., & Nguyen, K. C. (2025). *ViDRILL: A multi-stage retrieval framework for Vietnamese legal document search*. In *Proceedings of the 11th International Workshop on Vietnamese Language and Speech Processing (VLSP 2025)* (pp. 120–126). Hanoi, Vietnam: Association for Computational Linguistics. https://aclanthology.org/2025.vlsp-1.17/

[20] Culotta, A., & Sorensen, J. (2004). *Dependency tree kernels for relation extraction.* In Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics (pp. 423–429).

[21] Mintz, M., Bills, S., Snow, R., & Jurafsky, D. (2009). *Distant supervision for relation extraction without labeled data.* In Proceedings of ACL-IJCNLP 2009 (pp. 1003–1011).

[22] Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). *Self-attention with relative position representations.* In Proceedings of NAACL-HLT 2018 (pp. 464–468).

[23] Mai, V. T., Le, H.-Q., Can, D.-C., Nguyen, T. M. H., Nguyen, T. N. L., & Doan, T. T. (2020). *Overview of the VLSP RelEx shared task.* In Proceedings of the 7th International Workshop on Vietnamese Language and Speech Processing (pp. 92–98).

[24] Pham, Q. N. M. (2018). *A feature-rich Vietnamese named-entity recognition model.* arXiv preprint arXiv:1803.04375.

[25] Nguyen, D. Q., & Nguyen, A. T. (2020). *PhoBERT: Pre-trained language models for Vietnamese.* arXiv preprint arXiv:2003.00744.

[26] Vuong, T.-H.-Y., Hoang, M.-Q., Nguyen, T.-M., Nguyen, H.-T., & Nguyen, H.-T. (2023). *Constructing a knowledge graph for Vietnamese legal cases with heterogeneous graphs.* arXiv preprint arXiv:2309.09069.

[27] Ngo, H. Q., Nguyen, H. D., & Le-Khac, N.-A. (2024). *Ontology knowledge map approach towards building linked data for Vietnamese legal applications.* Vietnam Journal of Computer Science, 11(2), 323–342.

[28] de Martim, H. (2025). *An ontology-driven Graph RAG for legal norms.* arXiv preprint arXiv:2505.00039.

[29] Gerritse, E. J., Hasibi, F., & de Vries, A. P. (2025). *Graph embeddings to empower entity retrieval.* Information Retrieval Research, 1, 137–165.

[30] Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., & Tang, S. (2024). *Graph retrieval-augmented generation: A survey.* arXiv preprint arXiv:2408.08921.

[31] Raja, R., & Vats, A. (2025). *Beyond nearest neighbors: Semantic compression and graph-augmented retrieval.* arXiv preprint arXiv:2507.19715.

[32] Wang, X., Gao, T., Zhu, Z., Zhang, Z., Liu, Z., Li, J., & Tang, J. (2019). *KEPLER: A unified model for knowledge embedding and pre-trained language representation.* arXiv preprint arXiv:1911.06136.

[33] Li, S., Chen, G., Zhang, L., Jin, H., & Sun, G. (2025). *A multi-perspective knowledge graph embedding model.* Knowledge-Based Systems, 329, 114331.

[34] Trouillon, T., Welbl, J., Riedel, S., Gaussier, É., & Bouchard, G. (2016). *Complex embeddings for simple link prediction.* In Proceedings of ICML 2016 (pp. 2071–2080).

[35] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). *Inductive representation learning on large graphs.* arXiv preprint arXiv:1706.02216.

[36] Kipf, T. N., & Welling, M. (2016). *Semi-supervised classification with graph convolutional networks.* arXiv preprint arXiv:1609.02907.

[37] Kipf, T. N., & Welling, M. (2016). *Variational graph auto-encoders.* arXiv preprint arXiv:1611.07308.

[38] Knollmeyer, S., Caymazer, O., & Grossmann, D. (2025). *Document GraphRAG: Knowledge graph enhanced retrieval-augmented generation.* Electronics, 14(11), 2102.

[39] Li, Z., Guo, Q., Shao, J., Song, L., Bian, J., Zhang, J., & Wang, R. (2024). *Graph neural network enhanced retrieval for question answering of large language models*. arXiv preprint arXiv:2406.06572.

[40] Zhong, H., Guo, Z., Tu, C., Xiao, C., Liu, Z., & Sun, M. (2018). *Legal judgment prediction via topological learning*. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (pp. 3540–3549).

[41] Chalkidis, I., & Kampas, D. (2019). *Deep learning in law: early adaptation and legal word embeddings trained on large corpora*. Artificial Intelligence and Law, 27(2), 171–198. https://doi.org/10.1007/s10506-018-9238-9

[42] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv preprint arXiv:2005.11401.